

method used to direct packets from one station to just one other station or a subset of all stations will be discussed later. For now, it is enough to understand that the lowest level hardware does not distinguish among transmissions — a hub passes all packets to each host interface, which chooses packets the computer should receive and filters out all others. Ethernet is called a *best-effort delivery* mechanism because the hardware provides no information to the sender about whether the packet was delivered. For example, if the destination machine happens to be powered down, packets sent to it will be lost, and the sender will not be notified. We will see later how the TCP/IP protocols accommodate best-effort delivery hardware.

Ethernet access control is distributed because, unlike some network technologies, Ethernet has no central authority to grant access. The Ethernet access scheme is called *Carrier Sense Multiple Access with Collision Detect (CSMA/CD)*. It is *CSMA* because multiple machines can access an Ethernet simultaneously and each machine determines whether the network is idle by sensing whether a carrier wave is present. When a host interface has a packet to transmit, it listens to see if a message is being transmitted (i.e., performs carrier sensing). When no transmission is sensed, the host interface starts transmitting. Each transmission is limited in duration because there is a maximum packet size. Furthermore, the hardware must observe a minimum idle time between transmissions, which means that no single pair of communicating machines can use the network without giving other machines an opportunity for access.

2.4.8 Collision Detection And Recovery

When a station begins transmission, the signal does not reach all parts of the network simultaneously. Instead it travels along copper wires at approximately 70% of the speed of light. Thus, it is possible for two transceivers to both sense that the network is idle and begin transmission simultaneously. When the two electrical signals cross they become scrambled, meaning that neither remains meaningful. Such incidents are called *collisions*.

The Ethernet handles collisions in an ingenious fashion. Each station monitors the cable while it is transmitting to see if a foreign signal interferes with its transmission. Technically, the monitoring is called *collision detection (CD)*, making the Ethernet a CSMA/CD network. When a collision is detected, the host interface aborts transmission, waits for activity to subside, and tries again. Care must be taken or the network could wind up with all stations busily attempting to transmit and every transmission producing a collision. To help avoid such situations, Ethernet uses a binary exponential backoff policy where a sender delays a random time after the first collision, doubles the range if a second attempt to transmit also produces a collision, quadruples the range if a third attempt results in a collision, and so on. The motivation for exponential backoff is that in the unlikely event many stations attempt to transmit simultaneously, a severe traffic jam could occur. In such a jam, there is a high probability two stations will choose random backoffs that are close together. Thus, the probability of another collision is high. By doubling the range of the random delay, the exponential backoff strategy quickly spreads the stations' attempts to retransmit over a reasonably long period of time, making the probability of further collisions extremely small.

2.4.9 Ethernet Hardware Addresses

Ethernet defines a 48-bit addressing scheme. Each computer attached to an Ethernet network is assigned a unique 48-bit number known as its *Ethernet address*. To assign an address, Ethernet hardware manufacturers purchase blocks of Ethernet addresses† and assign them in sequence as they manufacture Ethernet interface hardware. Thus, no two hardware interfaces have the same Ethernet address.

Usually, the Ethernet address is fixed in machine readable form on the host interface hardware. Because each Ethernet address belongs to a hardware device, they are sometimes called *hardware addresses*, *physical addresses*, *media access (MAC) addresses*, or *layer 2 addresses*. Note the following important property of Ethernet physical addresses:

Physical addresses are associated with the Ethernet interface hardware; moving the hardware interface to a new machine or replacing a hardware interface that has failed changes the machine's physical address.

Knowing that Ethernet physical addresses can change will make it clear why higher levels of the network software are designed to accommodate such changes.

The host interface hardware examines packets and determines the packets that should be sent to the host. Recall that each interface receives a copy of every packet that passes through a hub — even those addressed to other machines. The host interface uses the destination address field in a packet as a filter. The interface ignores those packets that are addressed to other machines, and passes to the host only those packets addressed to it. The addressing mechanism and hardware filter are needed to prevent a computer from being overwhelmed with incoming data. Although the computer's central processor could perform the check, doing so in the host interface keeps traffic on the Ethernet from slowing down processing on all computers.

A 48-bit Ethernet address can do more than specify a single destination computer. An address can be one of three types:

- The physical address of one network interface (a *unicast address*)
- The network *broadcast address*
- A *multicast address*

By convention, the broadcast address (all 1s) is reserved for sending to all stations simultaneously. Multicast addresses provide a limited form of broadcast in which a subset of the computers on a network agree to listen to a given multicast address. The set of participating computers is called a *multicast group*. To join a multicast group, a computer must instruct its host interface to accept the group's multicast address. The advantage of multicasting lies in the ability to limit broadcasts: every computer in a multicast group can be reached with a single packet transmission, but computers that choose not to participate in a particular multicast group do not receive packets sent to the group.

†The Institute for Electrical and Electronic Engineers (IEEE) manages the Ethernet address space and assigns addresses as needed.

To accommodate broadcast and multicast addressing, Ethernet interface hardware must recognize more than its physical address. A computer interface usually accepts at least two kinds of packets: those addressed to the interface’s physical (i.e., unicast) address and those addressed to the network broadcast address. Some interfaces can be programmed to recognize multicast addresses or even alternate physical addresses. When a computer boots, the operating system initializes the Ethernet interface hardware, giving it a set of addresses to recognize. The interface then examines the destination address field in each packet, passing on to the computer only those transmissions designated for one of the specified addresses.

2.4.10 Ethernet Frame Format

Ethernet should be thought of as a link-level connection among machines. Thus, it makes sense to view the data transmitted as a *frame*†. Ethernet frames are of variable length, with no frame smaller than 64 octets‡ or larger than 1518 octets (header, data, and CRC). As in all packet-switched networks, each Ethernet frame contains a field that contains the address of its destination. Figure 2.7 shows that the Ethernet frame format contains the physical source address as well as the destination address.

Preamble	Destination Address	Source Address	Frame Type	Frame Data	CRC
8 octets	6 octets	6 octets	2 octets	46–1500 octets	4 octets

Figure 2.7 The format of a frame (packet) as it travels across an Ethernet preceded by a preamble. Fields are not drawn to scale.

In addition to identifying the source and destination, each frame transmitted across the Ethernet contains a *preamble*, *type field*, *data field*, and *Cyclic Redundancy Check (CRC)*. The preamble consists of 64 bits of alternating 0s and 1s to help receiving interfaces synchronize. The 32-bit CRC helps the interface detect transmission errors: the sender computes the CRC as a function of the data in the frame, and the receiver recomputes the CRC to verify that the packet has been received intact.

The frame type field contains a 16-bit integer that identifies the type of the data being carried in the frame. From the Internet point of view, the frame type field is essential because it means Ethernet frames are *self-identifying*. When a frame arrives at a given machine, the operating system uses the frame type to determine which protocol software module should process the frame. The chief advantages of self-identifying frames are that they allow multiple protocols to be used together on a single computer and they allow multiple protocols to be intermixed on the same physical network without interference. For example, one could have an application program on a computer using Internet protocols while another application on the same computer uses a local experimental protocol. The operating system examines the type field of each arriv-

†The term *frame* derives from communication over serial lines in which the sender “frames” the data by adding special characters before and after the transmitted data.

‡Technically, the term *byte* refers to a hardware-dependent character size; networking professionals use the term *octet*, because it refers to an 8-bit quantity on all computers.

ing frame to decide how to process the contents. We will see that the TCP/IP protocols use self-identifying Ethernet frames to distinguish among several protocols.

2.4.11 Extending An Ethernet With Repeaters

Although the original Ethernet cable had a maximum length, a network could be extended in two ways: using repeaters and bridges. An electronic device called a *repeater* operates on analog electrical signals. Like a hub in a twisted pair Ethernet, a repeater relays all electrical signals from one cable to another. Specifically, in the original thick Ethernet wiring scheme, a repeater can be placed between a pair of coaxial cables to double the total length. However, to preserve the CSMA/CD timing, the Ethernet standard restricts the use of repeaters — at most two repeaters can be placed between any two machines. Figure 2.8 shows a typical use of repeaters in an office building. A single cable runs vertically up the building, and a repeater attaches the backbone to an additional cable on each floor. Computers attach to the cables on each floor.

2.4.12 Extending An Ethernet With Bridges

Connecting two Ethernets with a bridge is superior to connecting them with a repeater or hub because bridges operate on packets rather than electrical signals. In particular, a bridge does not replicate noise, errors, or malformed frames; the bridge must receive a completely valid frame from one segment before the bridge will accept and transmit it on the other segment. Furthermore, each connection between a bridge and an Ethernet network follows the CSMA/CD rules, so collisions and propagation delays on one segment remain isolated from those on the other. As a result, an (almost) arbitrary number of Ethernets can be connected together with bridges. The important point is:

Bridges hide the details of interconnection: a set of bridged segments acts like a single Ethernet.

Bridged networks are classified as *transparent* because a computer does not know how many bridges connect segments of the network. The computer uses exactly the same hardware, frame format, and procedures to communicate with a computer across a bridge as it uses to communicate with a computer on the local segment.

Most bridges do much more than replicate frames from one wire to another: they make intelligent decisions about which frames to forward. Such bridges are called *adaptive* or *learning* bridges. An adaptive bridge consists of a computer with two Ethernet interfaces. The software in an adaptive bridge keeps two address lists, one for each interface. When a frame arrives from Ethernet E_i , the adaptive bridge adds the 48-bit Ethernet *source* address to the list associated with E_i . Similarly, when a frame

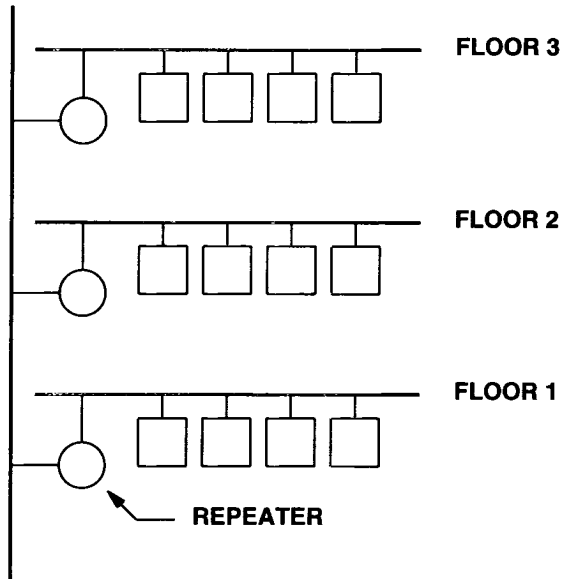


Figure 2.8 Repeaters used to join Ethernet cables in a building. At most two repeaters can be placed between a pair of communicating machines.

arrives from Ethernet E_2 , the bridge adds the source address to the list associated with E_2 . Thus, over time the adaptive bridge will learn which machines lie on E_1 and which lie on E_2 .

After recording the source address of a frame, the adaptive bridge uses the destination address to determine whether to forward the frame. If the address list shows that the destination lies on the Ethernet from which the frame arrived, the bridge does not forward the frame. If the destination is not in the address list (i.e., the destination is a broadcast or multicast address or the bridge has not yet learned the location of the destination), the bridge forwards the frame to the other Ethernet.

The advantages of adaptive bridges should be obvious. Because the bridge uses addresses found in normal traffic, it is completely automatic — humans need not configure the bridge with specific addresses. Because it does not forward traffic unnecessarily, a bridge helps improve the performance of an overloaded network by isolating traffic on specific segments. Bridges work exceptionally well if a network can be divided physically into two segments that each contain a set of computers that communicate frequently (e.g., each segment contains a set of workstations along with a server, and the workstations direct most of their traffic to the server). To summarize:

An adaptive Ethernet bridge connects two Ethernet segments, forwarding frames from one to the other. It uses source addresses to learn which machines lie on which Ethernet segment, and it combines information learned with destination addresses to eliminate forwarding when unnecessary.

From the TCP/IP point of view, bridged Ethernets are merely another form of physical network connection. The important point is:

Because the connection among physical cables provided by bridges and repeaters is transparent to machines using the Ethernet, we think of multiple Ethernet segments connected by bridges and repeaters as a single physical network system.

Most commercial bridges are much more sophisticated and robust than our description indicates. When first powered up, they check for other bridges and learn the topology of the network. They use a distributed spanning-tree algorithm to decide how to forward frames. In particular, the bridges decide how to propagate broadcast packets so only one copy of a broadcast frame is delivered to each wire. Without such an algorithm, Ethernets and bridges connected in a cycle would produce catastrophic results because they would forward broadcast packets in both directions simultaneously.

2.5 Fiber Distributed Data Interconnect (FDDI)

FDDI is another popular local area networking technology that provides a data rate of 100 Mbps (i.e., the same data rate as Fast Ethernet). Unlike Ethernet and other LAN technologies that use copper cables to carry electrical signals, FDDI is designed to use optical fiber. Data is encoded in pulses of light[†].

Optical fiber has two advantages over copper wire. First, because electrical noise does not interfere with an optical connection, the fiber can lie adjacent to powerful electrical devices. Second, because optical fibers use light, the amount of data that can be sent per unit time is much higher than cables that carry electrical signals.

It might seem that glass fibers would be difficult to install and would break if bent. However, an optical cable is surprisingly flexible. The glass fiber itself has an extremely small diameter, and the cable includes a plastic jacket that protects the fiber from breaking. Such a cable cannot bend at a ninety degree angle, but it can bend in an arc with a diameter of a few inches. Thus, installation is not difficult.

[†]A related technology known as *Copper Distributed Data Interface (CDDI)* works like FDDI, but uses copper cables to carry signals.

2.5.1 Properties Of An FDDI Network

An FDDI network is a 100 Mbps shared token passing ring technology with a self-healing capability. An FDDI network is *shared* because multiple computers connect to a given network and take turns sending packets. FDDI is known as a *ring* because the network forms a cycle that starts at one computer, passes through all others computers, and ends back at the source. FDDI is a *token passing ring* (or simply a *token ring*) technology because it uses token passing to control transmission. When the network is idle, a special, reserved frame called a *token* circulates around the ring from station to station. When a station has a packet to send, it waits for the token to arrive, sends its packet, and then passes the token to the next station. The circulating token guarantees fairness: it ensures that all stations have an opportunity to send a packet before any station sends a second packet.

Perhaps the most interesting property of an FDDI lies in its ability to detect and correct problems. The network is called *self-healing* because the hardware can automatically accommodate a failure.

2.5.2 Dual Counter-Rotating Rings

To provide automatic recovery from failures, FDDI hardware uses two independent rings that both connect to each computer. Figure 2.9 illustrates the topology.

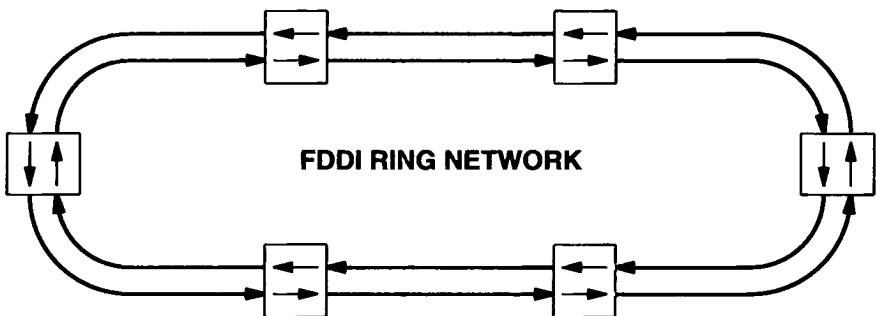


Figure 2.9 An FDDI network with optical fibers interconnecting six computers. Arrows show the direction of traffic on the fibers and through the attached computers.

FDDI rings are called *counter rotating* because traffic passes in the opposite direction on each ring. The reason for using a counter rotating scheme will become clear when we consider how FDDI handles failures.

Unless an error has occurred, an FDDI hardware does not need both rings. In fact, an FDDI interface behaves like any token passing network interface until an error occurs. The interface examines all packets that circulate around the ring, comparing the

destination address in each packet to the computer's address. The interface keeps a copy of any packet destined for the local computer, but also forwards the packet around the ring.

When a computer needs to transmit a packet, it waits for the token to arrive, temporarily stops forwarding bits, and sends its packet. After sending one packet, the interface transmits the token, and begins forwarding bits again. Even if a station has more than one packet ready to be sent when it receives the token, the station only sends one packet before passing the token.

FDDI hardware becomes more interesting when a hardware error occurs. When an interface detects that it cannot communicate with the adjacent computer, the interface uses the backup ring to bypass the failure. For example, Figure 2.10 shows an FDDI ring in which an interface has failed, and the two adjacent interfaces have eliminated it from the ring.

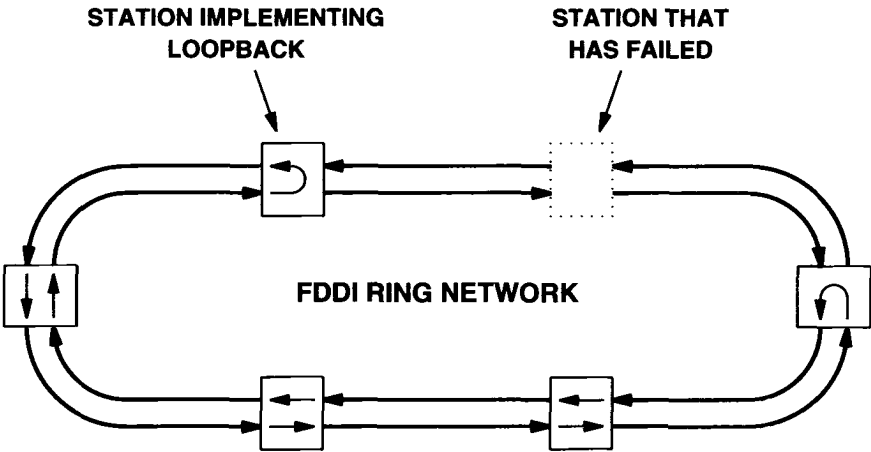


Figure 2.10 An FDDI ring after a failure. When FDDI hardware detects such a failure, it uses the second ring to bypass the failure and allows remaining stations to communicate.

The purpose of the second ring and the reason data flows in the opposite direction should now be clear: a failure can mean that the fiber has been disconnected (e.g., accidentally cut). If the fiber from both rings follows the same physical path, chances are high that the second fiber may have been disconnected as well. FDDI hardware automatically uses the counter rotating ring to form a closed loop in the direction that is still working. Doing so permits the other computers to continue communication despite the failure.

When FDDI hardware detects a failure on the network, it automatically loops data across the backup ring to permit communication among remaining stations.

2.5.3 FDDI Frame Format

FDDI standards specify the exact format of frames used on the network. The table in Figure 2.11 lists fields in an FDDI frame.

Field	Length in 4-bit units	Contents
PA	4 or more	Preamble
SD	2	Start Delimiter
FC	2	Frame Control
DA	4 or 12	Destination Address
SA	4 or 12	Source Address
RI	0 to 60	Routing Information
DATA	0 or more	Data
FCS	8	Frame Check Sequence
ED	1	End Delimiter
FS	3 or more	Frame Status

Figure 2.11 The format of frames used by FDDI, with fields measured in 4-bit units called *symbols*. The maximum frame length is 9000 symbols.

Like other technologies, each computer attached to an FDDI network is assigned an address, and each frame contains a destination address field. However, to make FDDI more flexible and to provide a standard way to interconnect two FDDI rings, the designers allowed more than one frame format. For example, the destination address field is either 4 or 12 symbols long, where a *symbol* is a 4-bit unit. The frame also includes a field used for routing. The sender can use the routing field to specify that a frame must be sent first to a connection point and then on to a destination on an attached ring.

One of the advantages of FDDI arises from its large frame size. Because a frame can contain 9000 4-bit symbols, the total frame can be 4500 octets long. Because header information occupies at most a few hundred octets, a single frame can carry 4K octets of user data. For applications that transfer large volumes of data (e.g., file transfer), the large frame size means less overhead and consequently high throughput.

2.6 Asynchronous Transfer Mode

Asynchronous Transfer Mode (ATM) is the name given to a connection-oriented networking technology that is intended for use in both local area and wide area networks. ATM is designed to permit extremely high speed data switching; the fastest ATM hardware can switch data at gigabit speeds†. Of course, such high speeds require complex, state-of-the-art hardware. As a result, ATM networks are more expensive than other technologies.

To achieve high transfer speeds, an ATM network uses special-purpose hardware and software techniques. First, an ATM network consists of one or more high-speed switches that each connect to computers and to other ATM switches. Second, ATM uses optical fibers for connections, including connections from a user's computer to an ATM switch. Optical fibers provide a higher transfer rate than copper wires; typically, the connection between a user's computer and an ATM switch operates at 155 Mbps. Third, the lowest layers of an ATM network use fixed-size frames called *cells*. Because each cell is exactly the same size, ATM switch hardware can process cells quickly.

2.6.1 ATM Cell Size

Surprisingly, each ATM cell is only 53 octets long. The cell contains 5 octets of header followed by 48 octets of data. Later chapters will show, however, that when using ATM to send IP traffic, the 53 octet size is irrelevant — an ATM network accepts and delivers much larger packets.

2.6.2 Connection-Oriented Networking

ATM differs from the packet-switching networks described earlier because it offers *connection-oriented* service. Before a computer connected to an ATM switch can send cells, a connection must be established manually or the host must first interact with the switch to specify a destination. The interaction is analogous to placing a telephone call‡. The requesting computer specifies the remote computer's address, and waits for the ATM switch to find a path through the network and establish a connection. If the remote computer rejects the request, does not respond, or the ATM switches between the sender and receiver cannot currently establish a path, the request to establish communication fails.

Once a connection succeeds, the local ATM switch chooses an identifier for the connection, and passes the connection identifier to the computer along with a message that informs the computer of success. The computer uses the connection identifier when sending or receiving cells.

When it finishes using a connection, the computer again communicates with the ATM switch to request that the connection be broken. The switch then disconnects the two computers. Disconnection is equivalent to hanging up a telephone at the end of a telephone call; after a disconnection, the computers cannot communicate until they es-

†Most computers cannot generate or absorb data at gigabit rates; ATM networks operate at gigabit speed to handle the traffic from many computers.

‡Because ATM was designed to carry voice as well as data, there is a strong relationship between an ATM network and a telephone system.

establish a new connection. Furthermore, identifiers used for a connection can be recycled; once a disconnection occurs, the switch can reuse the connection identifier for a new connection.

2.7 WAN Technologies: ARPANET

We will see that wide area networks have important consequences for internet addressing and routing. The technologies discussed in the remainder of this chapter were selected because they figure prominently in both the history of the Internet and later examples in the text.

One of the oldest wide area technologies, the ARPANET, was funded by *ARPA*, the *Advanced Research Projects Agency*. ARPA awarded a contract for the development of the ARPANET to Bolt, Beranek and Newman of Cambridge, MA in the fall of 1968. By September 1969, the first pieces of the ARPANET were in place.

The ARPANET served as a testbed for much of the research in packet-switching. In addition to its use for network research, researchers in several universities, military bases, and government labs regularly used the ARPANET to exchange files and electronic mail and to provide remote login among their sites. In 1975, control of the network was transferred from ARPA to the U.S. Defense Communications Agency (DCA). The DCA made the ARPANET part of the Defense Data Network (DDN), a program that provides multiple networks as part of a world-wide communication system for the Department of Defense.

In 1983, the Department of Defense partitioned the ARPANET into two connected networks, leaving the ARPANET for experimental research and forming the *MILNET* for military use. MILNET was restricted to unclassified data because it was not considered secure. Although under normal circumstances, both ARPANET and MILNET agreed to pass traffic to each other, controls were established that allowed them to be disconnected†. Because the ARPANET and MILNET used the same hardware technology, our description of the technical details apply to both. In fact, the technology was also available commercially and was used by several corporations to establish private packet switching networks.

Because the ARPANET was already in place and used daily by many of the researchers who developed the Internet architecture, it had a profound effect on their work. They came to think of the ARPANET as a dependable wide area backbone around which the Internet could be built. The influence of a single, central wide area backbone is still painfully obvious in some of the Internet protocols that we will discuss later, and has prevented the Internet from accommodating additional backbone networks gracefully.

Physically, the ARPANET consisted of approximately 50 BBN Corporation C30 and C300 minicomputers, called *Packet Switching Nodes* or *PSNs*‡ scattered across the continental U.S. and western Europe; MILNET contained approximately 160 PSNs, including 34 in Europe and 18 in the Pacific and Far East. One PSN resided at each site participating in the network and was dedicated to the task of switching packets; it could

†Perhaps the best known example of disconnection occurred in November 1988 when a *worm* program attacked the Internet and replicated itself as quickly as possible.

‡PSNs were initially called *Interface Message Processors* or *IMPs*; some publications still use the term IMP as a synonym for packet switch.

not be used for general-purpose computation. Indeed, each PSN was considered to be part of the ARPANET, and was owned and controlled by the *Network Operations Center (NOC)* located at BBN in Cambridge, Massachusetts.

Point-to-point data circuits leased from common carriers connected the PSNs together to form a network. For example, leased data circuits connected the ARPANET PSN at Purdue University to the ARPANET PSNs at Carnegie Mellon and at the University of Wisconsin. Initially, most of the leased data circuits in the ARPANET operated at 56 Kbps, a speed considered fast in 1968 but extremely slow by current standards. Remember to think of the network speed as a measure of capacity rather than a measure of the time it takes to deliver packets. As more computers used the ARPANET, capacity was increased to accommodate the load. For example, during the final year the ARPANET existed, many of the cross-country links operated over megabit-speed channels.

The idea of having no single point of failure in a system is common in military applications because reliability is important. When building the ARPANET, ARPA decided to follow the military requirements for reliability, so they mandated that each PSN had to have at least two leased line connections to other PSNs, and the software had to automatically adapt to failures and choose alternate routes. As a result, the ARPANET continued to operate even if one of its data circuits failed.

In addition to connections for leased data circuits, each ARPANET PSN had up to 22 *ports* that connected it to user computers, called *hosts*. Originally, each computer that accessed the ARPANET connected directly to one of the ports on a PSN. Normally, host connections were formed with a special-purpose interface board that plugged into the computer's I/O bus.

The original PSN port hardware used a complex protocol for transferring data across the ARPANET. Known as 1822, after the number of a technical report that described it, the protocol permitted a host to send a packet across the ARPANET to a specified destination PSN and a specified port on that PSN. Performing the transfer was complicated, however, because 1822 offered reliable, flow-controlled delivery. To prevent a given host from saturating the net, 1822 limited the number of packets that could be in transit. To guarantee that each packet arrived at its destination, 1822 forced the sender to await a *Ready For Next Message (RFNM)* signal from the PSN before transmitting each packet. The RFNM acted as an acknowledgement. It included a buffer reservation scheme that required the sender to reserve a buffer at the destination PSN before sending a packet.

Although there are many aspects not discussed here, the key idea is that underneath all the detail, the ARPANET was merely a transfer mechanism. When a computer connected to one port sent a packet to another port, the data delivered was exactly the data sent. Because the ARPANET did not provide a network-specific frame header, packets sent across it did not have a fixed field to specify packet type. Thus, unlike some network technologies, the ARPANET did not deliver self-identifying packets. In summary:

Networks such as the ARPANET or an ATM network do not have self-identifying frames. The attached computers must agree on the format and contents of packets sent or received to a specific destination.

Unfortunately, 1822 was never an industry standard. Because few vendors manufactured 1822 interface boards, it became difficult to connect new machines to the ARPANET. To solve the problem, ARPA later revised the PSN interface to use the X.25 standard[†]. The first version of an X.25 PSN implementation used only the data transfer part of the X.25 standard (known as HDLC/LAPB), but later versions made it possible to use all of X.25 when connecting to a PSN (i.e., ARPANET appeared to be an X.25 network).

Internally, of course, the ARPANET used its own set of protocols that were invisible to users. For example, there was a special protocol that allowed one PSN to request status from another, a protocol that PSNs used to send packets among themselves, and one that allowed PSNs to exchange information about link status and optimal routes.

Because the ARPANET was originally built as a single, independent network to be used for research, its protocols and addressing structure were designed without much thought given to expansion. By the mid 1970's, it became apparent no single network would solve all communication problems, and ARPA began to investigate satellite and packet radio network technologies. This experience with a variety of network technologies led to the concept of an internetwork.

2.7.1 ARPANET Addressing

While the details of ARPANET addressing are unimportant, they illustrate an alternative way in which wide area networks form physical addresses. Unlike the *flat address* schemes used by LAN technologies, wide area networks usually embed information in the address that helps the network route packets to their destination efficiently. In the ARPANET technology, each packet switch is assigned a unique integer, P , and each host port on the switch is numbered from 0 to $N-1$. Conceptually, a destination address consists of a pair of small integers, (P, N) . In practice, the hardware uses a single, large integer address, with some bits of the address used to represent N and others used to represent P .

2.8 National Science Foundation Networking

Realizing that data communication would soon be crucial to scientific research, in 1987 the National Science Foundation established a *Division of Network and Communications Research and Infrastructure* to help ensure that requisite network communications will be available for U.S. scientists and engineers. Although the division funds basic research in networking, its emphasis so far has been concentrated on providing seed funds to build extensions to the Internet.

[†]X.25 was standardized by the *Consultative Committee on International Telephone and Telegraph (CCITT)*, which later became the *Telecommunication Section of the International Telecommunication Union (ITU)*.

NSF's Internet extensions introduced a three-level hierarchy consisting of a U.S. backbone, a set of "mid-level" or "regional" networks that each span a small geographic area, and a set of "campus" or "access" networks. In the NSF model, mid-level networks attach to the backbone and campus networks attach to the mid-level nets. Each researcher had a connection from their computer to the local campus network. They used that single connection to communicate with local researchers' computers across the local campus net, and with other researchers further away. The campus network routed traffic across local nets to one of the mid-level networks, which routed it across the backbone as needed.

2.8.1 The Original NSFNET Backbone

Of all the NSF-funded networks, the NSFNET backbone has the most interesting history and used the most interesting technology. The backbone evolved in four major steps; it increased in size and capacity at the time the ARPANET declined until it became the dominant backbone in the Internet. The first version was built quickly, as a temporary measure. One early justification for the backbone was to provide scientists with access to NSF supercomputers. As a result, the first backbone consisted of six Digital Equipment Corporation LSI-11 microcomputers located at the existing NSF supercomputer centers. Geographically, the backbone spanned the continental United States from Princeton, NJ to San Diego, CA, using 56 Kbps leased lines as Figure 2.12 shows.

At each site, the LSI-11 microcomputer ran software affectionately known as *fuzzball*[†] code. Developed by Dave Mills, each fuzzball accessed computers at the local supercomputer center using a conventional Ethernet interface; it accessed leased lines leading to fuzzballs at other supercomputer centers using conventional link-level protocols over leased serial lines. Fuzzballs contained tables with addresses of possible destinations and used those tables to direct each incoming packet toward its destination.

The primary connection between the original NSFNET backbone and the rest of the Internet was located at Carnegie Mellon, which had both an NSFNET backbone node and an ARPANET PSN. When a user, connected to NSFNET, sent traffic to a site on the ARPANET, the packets would travel across the NSFNET to CMU where the fuzzball would route them onto the ARPANET via a local Ethernet. Similarly, the fuzzball understood that packets destined for NSFNET sites should be accepted from the Ethernet and sent across the NSF backbone to the appropriate site.

[†]The exact origin of the term "fuzzball" is unclear.

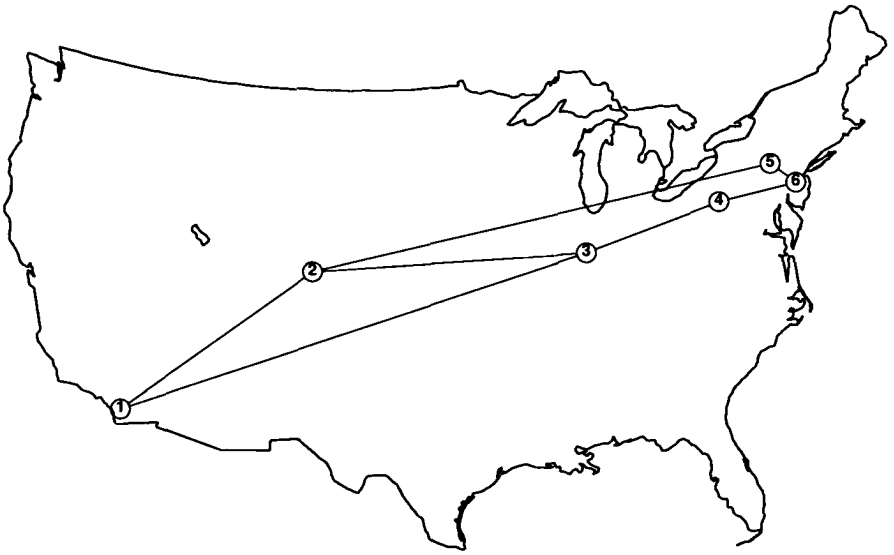


Figure 2.12 Circuits in the original NSFNET backbone with sites in (1) San Diego, CA; (2) Boulder, CO; (3) Champaign, IL; (4) Pittsburgh, PA; (5) Ithaca, NY; and (6) Princeton, NJ.

2.8.2 The Second NSFNET Backbone 1988-1989

Although users were excited about the possibilities of computer communication, the transmission and switching capacities of the original backbone were too small to provide adequate service. Within months after its inception, the backbone became overloaded and its inventor worked to engineer quick solutions for the most pressing problems, while NSF began the arduous process of planning for a second backbone.

In 1987, NSF issued a request for proposals from groups that were interested in establishing and operating a new, higher-speed backbone. Proposals were submitted in August of 1987 and evaluated that fall. On November 24, 1987 NSF announced it had selected a proposal submitted by a partnership of: MERIT Inc., the statewide computer network run out of the University of Michigan in Ann Arbor; IBM Corporation; and MCI Incorporated. The partners proposed to build a second backbone network, establish a network operation and control center in Ann Arbor, and have the system operational by the following summer. Because NSF had funded the creation of several new mid-level networks, the proposed backbone was designed to serve more sites than the original. Each additional site would provide a connection between the backbone and one of the NSF mid-level networks.

The easiest way to envision the division of labor among the three groups is to assume that MERIT was in charge of planning, establishing, and operating the network center. IBM contributed machines and manpower from its research labs to help MERIT develop, configure, and test needed hardware and software. MCI, a long-distance carrier, provided the communication bandwidth using the optical fiber already in place for its voice network. Of course, in practice there was close cooperation between all groups, including joint study projects and representatives from IBM and MCI in the project management.

By the middle of the summer of 1988, the hardware was in place and NSFNET began to use the second backbone. Shortly thereafter, the original backbone was shut down and disconnected. Figure 2.13 shows the logical topology of the second backbone after it was installed in 1988.

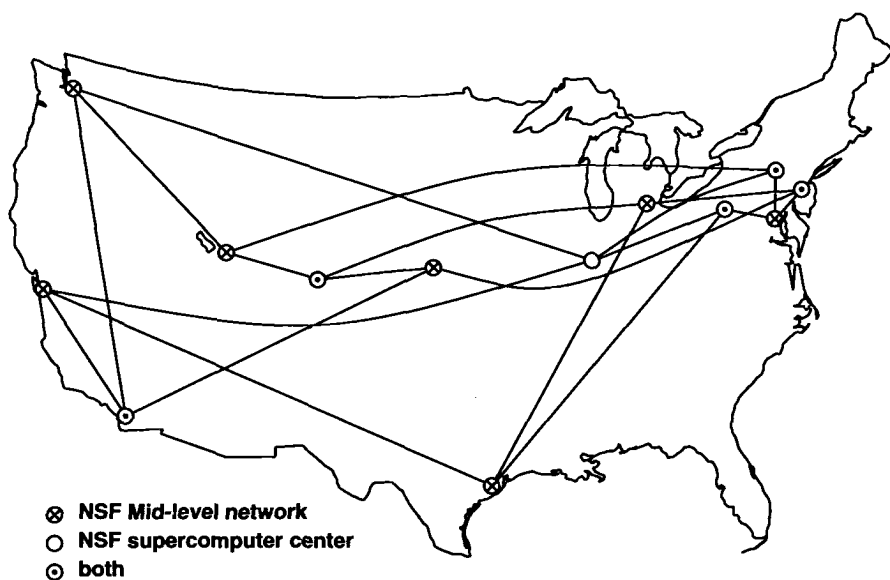


Figure 2.13 Logical circuits in the second NSFNET backbone from summer 1988 to summer 1989.

The technology chosen for the second NSFNET backbone was interesting. In essence, the backbone was a wide area network composed of packet routers interconnected by communication lines. As with the original backbone, the packet switch at each site connected to the site's local Ethernet as well as to communication lines leading to other sites.

2.8.3 NSFNET Backbone 1989-1990

After measuring traffic on the second NSFNET backbone for a year, the operations center reconfigured the network by adding some circuits and deleting others. In addition, they increased the speed of circuits to DS-1 (1.544 Mbps). Figure 2.14 shows the revised connection topology, which provided redundant connections to all sites.

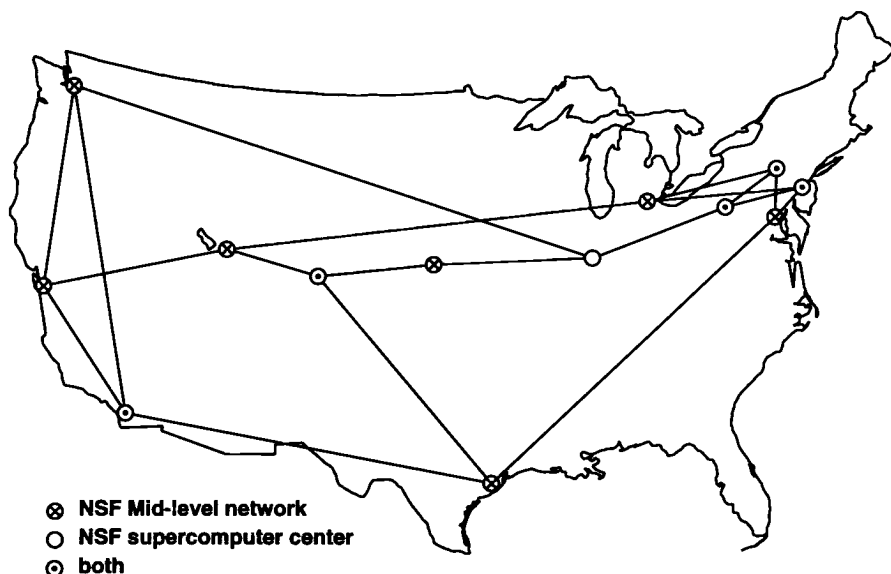


Figure 2.14 Circuits in the second NSFNET backbone from summer 1989 to 1990.

2.9 ANSNET

By 1991, NSF and other U.S. government agencies began to realize that the Internet was growing beyond its original academic and scientific domain. Companies around the world began to connect to the Internet, and nonresearch uses increased rapidly. Traffic on NSFNET had grown to almost one billion packets per day, and the 1.5 Mbps capacity was becoming insufficient for several of the circuits. A higher capacity backbone was needed. As a result, the U.S. government began a policy of commercialization and privatization. NSF decided to move the backbone to a private company and to charge institutions for connections.

Responding to the new government policy in December of 1991, IBM, MERIT, and MCI formed a not-for-profit company named *Advanced Networks and Services* (ANS). ANS proposed to build a new, higher speed Internet backbone. Unlike previous

wide area networks used in the Internet which had all been owned by the U.S. government, ANS would own the new backbone. By 1993, ANS had installed a new network that replaced NSFNET. Called *ANSNET*, the backbone consisted of data circuits operating at 45 Mbps†, giving it approximately 30 times more capacity than the previous NSFNET backbone. Figure 2.15 shows major circuits in ANSNET and a few of the sites connected in 1994. Each point of presence represents a location to which many sites connect.

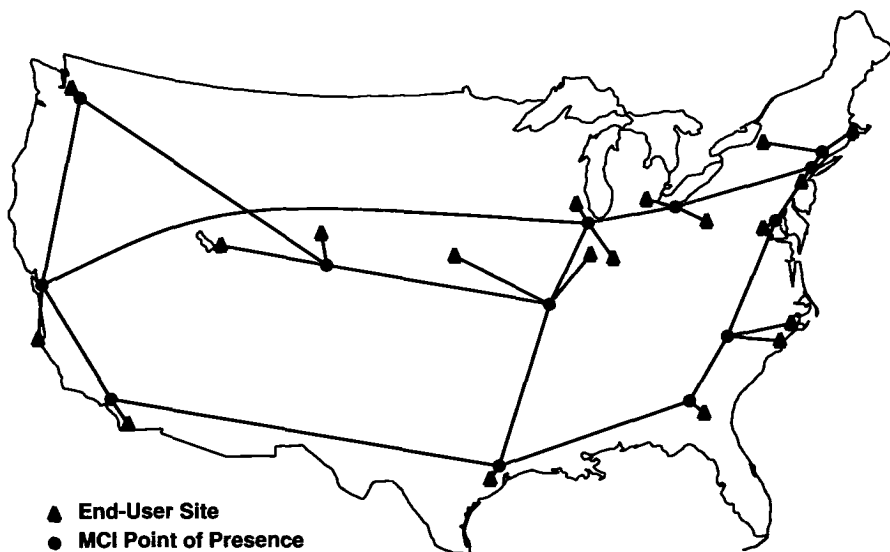


Figure 2.15 Circuits in ANSNET, the backbone of the U.S. Internet starting in 1993. Each circuit operates at 45 Mbps.

2.10 A Very High Speed Backbone (vBNS)

In 1995, NSF awarded MCI a contract to build a backbone operating at 155 Mbps (OC3 speed) to replace ANSNET. Called the *very high speed Backbone Network Service (vBNS)*, the new backbone offered a substantial increase in capacity, and required higher speed processors to route packets.

2.10.1 Commercial Internet Backbones

Since 1995, the Internet has become increasingly commercial, with the percentage of funding from the U.S. government steadily decreasing. Although vBNS still exists, it is now devoted to networking research. In its place, commercial companies have created large privately-funded backbones that carry Internet traffic. For example, public car-

†Telecommunication carriers use the term *DS3* to denote a circuit that operates at 45 Mbps; the term is often confused with *T3*, which denotes a specific encoding used over a circuit operating at DS3 speed.

riers like *AT&T* and *MCI* have each created large, high-capacity backbone networks used to carry Internet traffic from their customers. As discussed later, commercial backbones are interconnected through *peering arrangements*, making it possible for a customer of one company to send packets to a customer of another.

2.11 Other Technologies Over Which TCP/IP Has Been Used

One of the major strengths of TCP/IP lies in the variety of physical networking technologies over which it can be used. We have already discussed several widely used technologies, including local area and wide area networks. This section briefly reviews others that help illustrate an important principle:

Much of the success of the TCP/IP protocols lies in their ability to accommodate almost any underlying communication technology.

2.11.1 X25NET And Tunnels

In 1980, NSF formed the *Computer Science NETwork (CSNET)* organization to help provide Internet services to industry and small schools. CSNET used several technologies to connect its subscribers to the Internet, including one called *X25NET*. Originally developed at Purdue University, X25NET ran TCP/IP protocols over *Public Data Networks (PDNs)*. The motivation for building such a network arose from the economics of telecommunications: although leased serial lines were expensive, common carriers had begun to offer public packet-switched services. X25NET was designed to allow a site to use its connection to a public packet-switched service to send and receive Internet traffic.

Readers who know about public packet-switched networks may find X25NET strange because public services use the CCITT X.25 protocols exclusively while the Internet uses TCP/IP protocols. Unlike most packet switching hardware, X.25 protocols use a connection-oriented paradigm; like ATM, they were designed to provide connection-oriented service to individual applications. Thus, the use of X.25 to transport TCP/IP traffic foreshadowed the ways TCP/IP would later be transferred across ATM.

We have already stated that many underlying technologies can be used to carry Internet traffic, and X25NET illustrates how TCP/IP has been adapted to use high level facilities. The technique, sometimes called *tunneling*, simply means that TCP/IP treats a complex network system with its own protocols like any other hardware delivery system. To send TCP/IP traffic through an X.25 *tunnel*, a computer forms an X.25 connection and then sends TCP/IP packets as if they were data. The X.25 system carries the packets along its connection and delivers them to another X.25 endpoint, where they must be picked up and forwarded on to their ultimate destination. Because tunneling treats IP packets like data, the tunnel does not provide for self-identifying frames.

Thus, tunneling only works when both ends of the X.25 connection agree *a priori* that they will exchange IP packets (or agree on a format for encoding type information along with each packet).

Its connection-oriented interface makes X.25 even more unusual. Unlike connectionless networks, connection-oriented systems use a *virtual circuit* (VC) abstraction. Before data can be sent, switches in the network must set up a VC (i.e., a “path”) between the sender and the receiver. We said that the Internet protocols were optimized to run over a connectionless packet delivery system, which means that extra effort is required to run them over a connection-oriented network.

In theory, a single connection suffices for a tunnel through a connection-oriented network — after a pair of computers has established a VC, that pair can exchange TCP/IP traffic. In practice, however, the design of the protocols used on the connection-oriented system can make a single connection inefficient. For example, because X.25 protocols limit the number of packets that can be sent on a connection before an acknowledgement is received, such networks exhibit substantially better throughput when data is sent across multiple connections simultaneously. Thus, instead of opening a single connection to a given destination, X25NET improved performance by arranging for a sender to open multiple VCs and distribute traffic among them. A receiver must accept packets arriving on all connections, and combine them together again.

Tunneling across a high-level network such as X.25 requires mapping between the addresses used by the internet and addresses used by the network. For example, consider the addressing scheme used by X.25 networks, which is given in a related standard known as X.121. Physical addresses each consist of a 14-digit number, with 10 digits assigned by the vendor that supplies the X.25 network service. Resembling telephone numbers, one popular vendor’s assignment includes an area code based on geographic location. The addressing scheme is not surprising because it comes from an organization that determines international telephone standards. There is no mathematical relationship between such addresses and the addresses used by TCP/IP. Thus, a computer that tunnels TCP/IP data across an X.25 network must maintain a table of mappings between internet addresses and X.25 network addresses. Chapter 5 discusses the address mapping problem in detail and gives an alternative to using fixed tables. Chapter 18 shows that exactly the same problem arises for ATM networks, which use yet another alternative.

Because public X.25 networks operated independently of the Internet, a point of contact was needed between the two. Both ARPA and CSNET operated dedicated machines that provided the interconnection between X.25 and the ARPANET. The primary interconnection was known as the *VAN gateway*. The VAN agreed to accept X.25 connections and route each datagram that arrived over such a connection to its destination.

X25NET was significant because it illustrated the flexibility and adaptability of the TCP/IP protocols. In particular, it showed that tunneling makes it possible to use an extremely wide range of complex network technologies in an internet.

2.11.2 Point-To-Point Networks

We said that Wide Area Networks are usually composed of dedicated packet switches interconnected by data circuits leased from a telephone company. Phone companies originally designed such circuits to carry digitized voice calls; only later did their use in data networks become important. Consequently, the data rates of available circuits are not powers of ten. Instead, they have been chosen to carry multiples of 64 Kbps because a digitized voice call uses an encoding known as *Pulse Code Modulation (PCM)*, which produces 8000 samples per second, where each sample is 8 bits.

The table in Figure 2.16 lists a few common data rates used in North America and Europe.

Name	Bit Rate	Voice Circuits	Location
—	0.064 Mbps	1	
T1	1.544 Mbps	24	North America
T2	6.312 Mbps	96	North America
T3	44.736 Mbps	672	North America
E1	2.048 Mbps	30	Europe
E2	8.448 Mbps	120	Europe
E3	34.368 Mbps	480	Europe

Figure 2.16 Example data rates available on digital circuits leased from a telephone company. The rates were chosen to encode multiple voice calls.

Higher rate digital circuits are also available. In addition to standards that specify the transmission of high data rates over copper, the phone companies have developed standards for transmission of the same rates over optical fiber. The table in Figure 2.17 contains examples. Of course, circuits that operate at such high data rates are considerably more expensive than circuits that operate at lower rates.

Standard Name	Optical Name	Bit Rate	Voice Circuits
STS-1	OC-1	51.840 Mbps	810
STS-3	OC-3	155.520 Mbps	2430
STS-12	OC-12	622.080 Mbps	9720
STS-24	OC-24	1,244.160 Mbps	19440
STS-48	OC-48	2,488.320 Mbps	38880

Figure 2.17 Example data rates of high-capacity circuits that can be leased from phone companies. Optical fiber is used to achieve such high rates over long distances.

From TCP/IP's point of view, any communication system that connects exactly two computers is known as a *point-to-point network*. Thus, a leased data circuit between two computers is an example of a point-to-point network. Of course, using the term "network" to describe a connection between two computers stretches the concept. However, we will learn that viewing a connection as a network helps maintain consistency. For now, we only need to note that a point-to-point network differs from conventional networks in one significant way: because only two computers attach, no hardware addresses are used. When we discuss internet address binding, the lack of hardware addresses will make point-to-point networks an exception.

2.11.3 Dial-up IP

Another interesting use of TCP/IP pioneered by CSNET involves running TCP/IP protocols over the dial-up voice network (i.e., the telephone system). CSNET member sites that used the Internet infrequently could not justify the cost of a leased line connection. For such sites, CSNET developed a dial-up IP system that worked as expected: whenever a connection was needed, software at the member's site used a modem to form a connection to the CSNET hub over the voice telephone network. A computer at the hub answered the phone call and, after obtaining valid authorization, began to forward traffic between the site and other computers on the Internet. Dialing introduced a delay after the first packet was sent. However, for automated services like electronic mail, the delay was unnoticeable.

Dialup internet access provides another example of a point-to-point network. From the TCP/IP view, dialing a telephone call is equivalent to running a wire. Once the call has been answered by a modem on the other end, there is a connection from one computer directly to another, and the connection stays in place as long as needed.

2.11.4 Other Token Ring Technologies

FDDI is not the first token ring network technology; token ring products have existed for nearly twenty years. For example, IBM produces a popular token ring LAN technology. Early versions of the IBM token ring operated at 4 Mbps; later versions operate at 16 Mbps. Like other token ring systems, an IBM token ring network consists of a loop that attaches to all computers. A station must wait for a token before transmitting, and sends the token along after transferring a packet.

An older token ring technology designed by Proteon Corporation employs a novel hardware addressing scheme that will be used in a later chapter to illustrate one of the ways TCP/IP uses hardware addresses. Called a *proNET* network, the technology permits customers to choose a hardware address for each computer. Unlike an Ethernet, in which each interface board contains a unique address assigned by the manufacturer, a proNET interface board contains eight switches that must be set before the interface is installed in a computer. The switches form a number in binary between 0 and 255, inclusive. A given proNET network could have at most 254 computers attached because address 255 was reserved for broadcast and address 0 was not used. When first instal-

ling a proNET network, a network administrator chose a unique address for each computer. Typically, addresses were assigned sequentially, starting with 1.

A technology that permits customers to assign hardware addresses has advantages and disadvantages. The chief disadvantage arises from the potential for problems that occur if a network administrator accidentally assigns the same address to two computers. The chief advantage arises from ease of maintenance: if an interface board fails, it can be replaced without changing the computer's hardware address.

2.11.5 Wireless Network Technologies

One of the most interesting ARPA experiments in packet switching resulted in a *packet radio* technology that uses broadcast radio waves to carry packets. Designed for a military environment in which stations might be mobile, packet radio includes hardware and software that allow sites to find other sites, establish point-to-point communication, and then use the point-to-point communication to carry packets. Because sites change geographic location and may move out of communication range, the system must constantly monitor connectivity and recompute routes to reflect changes in topology. An operational packet radio system was built and used to demonstrate TCP/IP communication between a remote packet radio site and other sites on the Internet.

In recent years, a wide variety of wireless networking equipment has become available commercially. Wireless LANs use spread spectrum techniques such as direct sequencing or frequency hopping to provide data connections among a set of computers inside a building. The transmitters and antennas for such equipment are small and lightweight. The equipment can be attached to a portable notebook computer, making it convenient to move around an area such as an office building while remaining in communication.

Wireless broadband technology, originally developed as an alternative to cable television, is being used to transmit data. Known as *Multichannel Multipoint Distribution System (MMDS)*, the scheme has sufficient capacity to provide data rates as fast as those offered by the popular *Digital Subscriber Line (DSL)* technologies that deliver high data rates over copper telephone wires.

Cellular technology, which was originally designed for voice networks, has also been adapted to carry data. The chief advantage of a cellular system is the speed with which it allows users to move. Because the technology was designed to maintain voice communication even if a user travels by car, the underlying hardware can easily maintain contact with a mobile unit while transferring a stream of packets.

2.12 Summary And Conclusion

We have reviewed several network hardware technologies used by the TCP/IP protocols, ranging from inexpensive Local Area Network technologies like Ethernet and FDDI to expensive Wide Area Network technologies that use leased digital circuits to provide backbones. We have also seen that it is possible to run the TCP/IP protocols

over other general-purpose network protocols using a technique called tunneling. While the details of specific network technologies are not important, a general idea has emerged:

The TCP/IP protocols are extremely flexible; almost any underlying technology can be used to transfer TCP/IP traffic.

FOR FURTHER STUDY

Early computer communication systems employed point-to-point interconnection, often using general-purpose serial line hardware that McNamara [1982] describes. Metcalf and Boggs [1976] introduces the Ethernet with a 3 Mbps prototype version. Digital *et. al.* [1980] specifies the original 10 Mbps Ethernet standard, with IEEE standard 802.3 reported in Nelson [1983]. Shoch, Dalal, and Redell [1982] provides an historical perspective of the Ethernet evolution. Related work on the ALOHA network is reported in Abramson [1970], with a survey of technologies given by Cotton [1979].

Token passing ring technology is proposed in Farmer and Newhall [1969]. Miller and Thompson [1982], as well as Andrews and Shultz [1982], provide summaries. Another alternative, the slotted ring network, is proposed by Pierce [1972]. For a comparison of technologies, see Rosenthal [1982].

For more information on the ARPANET see Cerf [1989] and BBN [1981]. The ideas behind X25NET are summarized in Comer and Korb [1983]; Lanzillo and Partridge [January 1989] describes dial-up IP. De Prycker [1993] describes Asynchronous Transfer Mode and its use for wide area services. Partridge [1994] surveys many gigabit technologies, including ATM, and describes the internal structure of high speed switches.

EXERCISES

- 2.1 Find out which network technologies your site uses.
- 2.2 What is the maximum size packet that can be sent on a high-speed network like Network System Corporation's Hyperchannel?
- 2.3 If your site uses Ethernet hub technology, find out how many connections can be attached to a single hub. If your site has multiple hubs (e.g., one on each floor of a building), find out how the hubs communicate.
- 2.4 What are the advantages and disadvantages of tunneling?
- 2.5 Read the Ethernet standard to find exact details of the inter-packet gap and preamble size. What is the maximum steady-state rate at which Ethernet can transport data?

- 2.6** What characteristic of a satellite communication channel is most desirable? Least desirable?
- 2.7** Find a lower bound on the time it takes to transfer a 5 megabyte file across a network that operates at: 28.8 Kbps, 1.54 Mbps, 10 Mbps, 100 Mbps, and 2.4 Gbps.
- 2.8** Does the processor, disk, and internal bus on your computer operate fast enough to send data from a disk file at 2 gigabits per second?

3

Internetworking Concept And Architectural Model

3.1 Introduction

So far we have looked at the low-level details of transmission across individual data networks, the foundation on which all computer communication is built. This chapter makes a giant conceptual leap by describing a scheme that allows us to collect the diverse network technologies into a coordinated whole. The primary goal is a system that hides the details of underlying network hardware while providing universal communication services. The primary result is a high-level abstraction that provides the framework for all design decisions. Succeeding chapters show how we use this abstraction to build the necessary layers of internet communication software and how the software hides the underlying physical transport mechanisms. Later chapters also show how applications use the resulting communication system.

3.2 Application-Level Interconnection

Designers have taken two different approaches to hiding network details, using application programs to handle heterogeneity or hiding details in the operating system. Early heterogeneous network interconnections provided uniformity through application-level programs called *application gateways*. In such systems, an application-level program, executing on each computer in the network, understands the details of the network connections for that computer, and interoperates across those connections with application programs on other computers. For example, some electronic mail systems

consist of mail programs that are each configured to forward a memo to a mail program on the next computer. The path from source to destination may involve many different networks, but that does not matter as long as the mail systems on all the machines cooperate by forwarding each message.

Using application programs to hide network details may seem natural at first, but such an approach results in limited, cumbersome communication. Adding new functionality to the system means building a new application program for each computer. Adding new network hardware means modifying existing programs (or creating new programs) for each possible application. On a given computer, each application program must understand the network connections for the computer, resulting in duplication of code.

Users who are experienced with networking understand that once the interconnections grow to hundreds or thousands of networks, no one can possibly build all the necessary application programs. Furthermore, success of the step-at-a-time communication scheme requires correctness of all application programs executing along the path. When an intermediate program fails, the source and destination remain unable to detect or control the problem. Thus, systems that use intermediate applications programs cannot guarantee reliable communication.

3.3 Network-Level Interconnection

The alternative to providing interconnection with application-level programs is a system based on network-level interconnection. A network-level interconnection provides a mechanism that delivers small packets of data from their original source to their ultimate destination without using intermediate application programs. Switching small units of data instead of files or large messages has several advantages. First, the scheme maps directly onto the underlying network hardware, making it extremely efficient. Second, network-level interconnection separates data communication activities from application programs, permitting intermediate computers to handle network traffic without understanding the applications that are sending or receiving it. Third, using network connections keeps the entire system flexible, making it possible to build general purpose communication facilities. Fourth, the scheme allows network managers to add new network technologies by modifying or adding a single piece of new network level software, while application programs remain unchanged.

The key to designing universal network-level interconnection can be found in an abstract communication system concept known as *internetworking*. The internetwork, or *internet*, concept is an extremely powerful one. It detaches the notions of communication from the details of network technologies and hides low-level details from the user. More important, it drives all software design decisions and explains how to handle physical addresses and routes. After reviewing basic motivations for internetworking, we will consider the properties of an internet in more detail.

We begin with two fundamental observations about the design of communication systems:

- No single network hardware technology can satisfy all constraints.
- Users desire universal interconnection.

The first observation is an economic as well as technical one. Inexpensive Local Area Networks that provide high speed communication only cover short distances; wide area networks that span long distances cannot supply local communication cheaply. Because no single network technology satisfies all needs, we are forced to consider multiple underlying hardware technologies.

The second observation is self-evident. Ultimately, users would like to be able to communicate between any two points. In particular, we desire a communication system that is not constrained by the boundaries of physical networks.

The goal is to build a unified, cooperative interconnection of networks that supports a universal communication service. Within each network, computers will use underlying technology-dependent communication facilities like those described in Chapter 2. New software, inserted between the technology-dependent communication mechanisms and application programs, will hide the low-level details and make the collection of networks appear to be a single large network. Such an interconnection scheme is called an *internetwork* or *internet*.

The idea of building an internet follows a standard pattern of system design: researchers imagine a high-level computing facility and work from available computing technology, adding layers of software until they have a system that efficiently implements the imagined high-level facility. The next section shows the first step of the design process by defining the goal more precisely.

3.4 Properties Of The Internet

The notion of universal service is important, but it alone does not capture all the ideas we have in mind for a unified internet because there can be many implementations of universal services. In our design, we want to hide the underlying internet architecture from the user. That is, we do not want to require users or application programs to understand the details of hardware interconnections to use the internet. We also do not want to mandate a network interconnection topology. In particular, adding a new network to the internet should not mean connecting to a centralized switching point, nor should it mean adding direct physical connections between the new network and all existing networks. We want to be able to send data across intermediate networks even though they are not directly connected to the source or destination computers. We want all computers in the internet to share a universal set of machine identifiers (which can be thought of as *names* or *addresses*).

Our notion of a unified internet also includes the idea of network independence in the user interface. That is, we want the set of operations used to establish communication or to transfer data to remain independent of the underlying network technologies and the destination computer. Certainly, a user should not have to understand the network interconnection topology when creating or using application programs that communicate.

3.5 Internet Architecture

We have seen how computers connect to individual networks. The question arises, “How are networks interconnected to form an internetwork?” The answer has two parts. Physically, two networks can only be connected by a computer that attaches to both of them. A physical attachment does not provide the interconnection we have in mind, however, because such a connection does not guarantee that the computer will cooperate with other machines that wish to communicate. To have a viable internet, we need special computers that are willing to transfer packets from one network to another. Computers that interconnect two networks and pass packets from one to the other are called *internet gateways* or *internet routers*[†].

Consider an example consisting of two physical networks shown in Figure 3.1. In the figure, router *R* connects to both network 1 and network 2. For *R* to act as a router, it must capture packets on network 1 that are bound for machines on network 2 and transfer them. Similarly, *R* must capture packets on network 2 that are destined for machines on network 1 and transfer them.

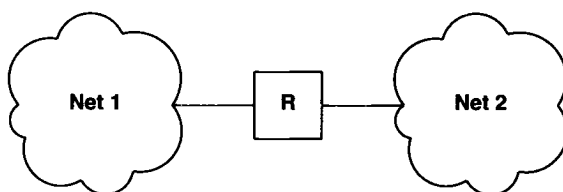


Figure 3.1 Two physical networks interconnected by *R*, a router (IP gateway).

In the figure, clouds are used to denote physical networks because the exact hardware is unimportant. Each network can be a LAN or a WAN, and each may have many computers attached or a few computers attached.

3.6 Interconnection Through IP Routers

Although it illustrates the basic connection strategy, Figure 3.1 is quite simplistic. In an actual internet that includes many networks and routers, each router needs to know about the topology of the internet beyond the networks to which it connects. For example, Figure 3.2 shows three networks interconnected by two routers.

[†]The original literature used the term *IP gateway*. However, vendors have adopted the term *IP router* — the two terms are used interchangeably throughout this text.

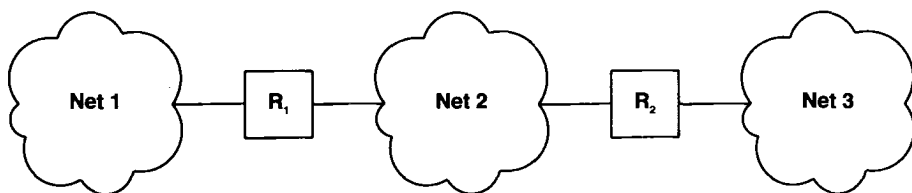


Figure 3.2 Three networks interconnected by two routers.

In this example, router R_1 must transfer from network 1 to network 2 all packets destined for computers on either network 2 or network 3. For a large internet composed of many networks, the router's task of making decisions about where to send packets becomes more complex.

The idea of a router seems simple, but it is important because it provides a way to interconnect networks, not just computers. In fact, we have already discovered the principle of interconnection used throughout an internet:

In a TCP/IP internet, special computers called IP routers or IP gateways provide interconnections among physical networks.

You might suspect that routers, which must each know how to forward packets toward their destination, are large machines with enough primary or secondary memory to hold information about every computer in the internet to which they attach. In fact, routers used with TCP/IP internets are usually small computers. They often have little disk storage and modest main memories. The trick to building a small internet router lies in the following concept:

Routers use the destination network, not the destination computer, when forwarding a packet.

If packet forwarding is based on networks, the amount of information that a router needs to keep is proportional to the number of networks in the internet, not the number of computers.

Because routers play a key role in internet communication, we will return to them in later chapters and discuss the details of how they operate and how they learn about routes. For now, we will assume that it is possible and practical to have correct routes for all networks in each router in the internet. We will also assume that only routers provide connections between physical networks in an internet.

3.7 The User's View

Remember that TCP/IP is designed to provide a universal interconnection among computers independent of the particular networks to which they attach. Thus, we want a user to view an internet as a single, virtual network to which all machines connect despite their physical connections. Figure 3.3a shows how thinking of an internet instead of constituent networks simplifies the details and makes it easy for the user to conceptualize communication. In addition to routers that interconnect physical networks, software is needed on each computer to allow application programs to use an internet as if it were a single, physical network.

The advantage of providing interconnection at the network level now becomes clear. Because application programs that communicate over the internet do not know the details of underlying connections, they can be run without change on any computer. Because the details of each machine's physical network connections are hidden in the internet software, only the internet software needs to change when new physical connections are added or existing connections are removed. In fact, it is possible to optimize the internal structure of the internet by altering physical connections while application programs are executing.

A second advantage of having communication at the network level is more subtle: users do not have to understand, remember, or specify how networks connect or what traffic they carry. Application programs can be written that communicate independent of underlying physical connectivity. In fact, network managers are free to change interior parts of the underlying internet architecture without changing application software in most of the computers attached to the internet (of course, network software must be reconfigured when a computer moves to a new network).

As Figure 3.3b shows, routers do not provide direct connections among all pairs of networks. It may be necessary for traffic traveling from one computer to another to pass through several routers as the traffic crosses intermediate networks. Thus, networks participating in an internet are analogous to highways in the U.S. interstate system: each net agrees to handle transit traffic in exchange for the right to send traffic throughout the internet. Typical users are unaffected and unaware of extra traffic on their local network.

3.8 All Networks Are Equal

Chapter 2 reviewed examples of the network hardware used to build TCP/IP internets, and illustrated the great diversity of technologies. We have described an internet as a collection of cooperative, interconnected networks. It is now important to understand a fundamental concept: from the internet point of view, any communication system capable of transferring packets counts as a single network, independent of its delay and throughput characteristics, maximum packet size, or geographic scale. In particular, Figure 3.3b uses the same small cloud shape to depict each physical network because TCP/IP treats them equally despite their differences. The point is:

The TCP/IP internet protocols treat all networks equally. A Local Area Network like an Ethernet, a Wide Area Network used as a backbone, or a point-to-point link between two computers each count as one network.

Readers unaccustomed to internet architecture may find it difficult to accept such a simplistic view of networks. In essence, TCP/IP defines an abstraction of “network” that hides the details of physical networks; we will learn that such abstractions help make TCP/IP extremely powerful.

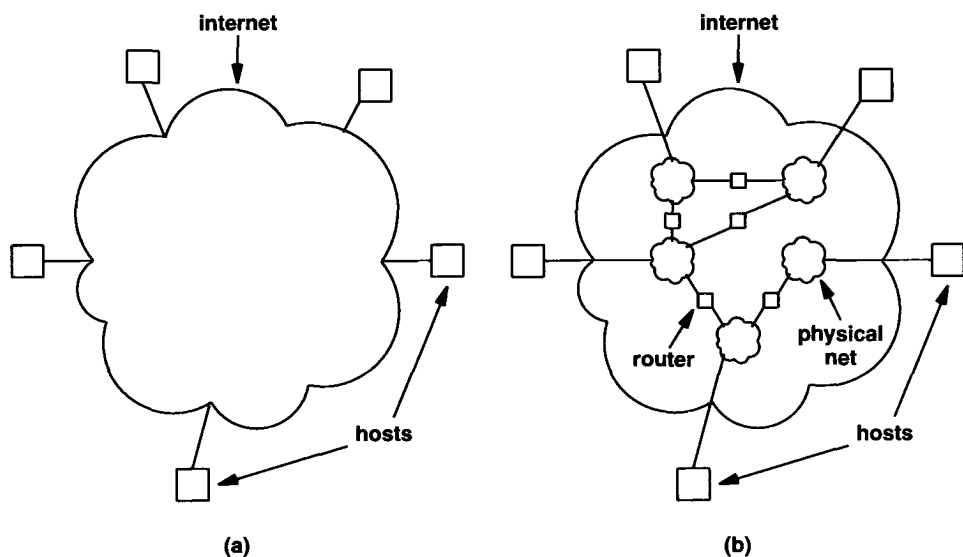


Figure 3.3 (a) The user's view of a TCP/IP internet in which each computer appears to attach to a single large network, and (b) the structure of physical networks and routers that provide interconnection.

3.9 The Unanswered Questions

Our sketch of internets leaves many unanswered questions. For example, you might wonder about the exact form of internet addresses assigned to computers or how such addresses relate to the Ethernet, FDDI, or ATM physical hardware addresses described in Chapter 2. The next three chapters confront these questions. They describe the format of IP addresses and illustrate how software on a computer maps between internet addresses and physical addresses. You might also want to know exactly what a packet looks like when it travels through an internet, or what happens when packets arrive too fast for some computer or router to handle. Chapter 7 answers these

questions. Finally, you might wonder how multiple application programs executing concurrently on a single computer can send and receive packets to multiple destinations without becoming entangled in each other's transmissions or how internet routers learn about routes. All of these questions will be answered as well.

Although it may seem vague now, the direction we are following will let us learn about both the structure and use of internet protocol software. We will examine each part, looking at the concepts and principles as well as technical details. We began by describing the physical communication layer on which an internet is built. Each of the following chapters will explore one part of the internet software, until we understand how all the pieces fit together.

3.10 Summary

An internet is more than a collection of networks interconnected by computers. Internetworking implies that the interconnected systems agree to conventions that allow each computer to communicate with every other computer. In particular, an internet will allow two computers to communicate even if the communication path between them passes across a network to which neither connects directly. Such cooperation is only possible when computers agree on a set of universal identifiers and a set of procedures for moving data to its final destination.

In an internet, interconnections among networks are formed by computers called IP routers, or IP gateways, that attach to two or more networks. A router forwards packets between networks by receiving them from one network and sending them to another.

FOR FURTHER STUDY

Our model of an internetwork comes from Cerf and Cain [1983] and Cerf and Kahn [1974], which describe an internet as a set of networks interconnected by routers and sketch an internet protocol similar to that eventually developed for the TCP/IP protocol suite. More information on the connected Internet architecture can be found in Postel [1980]; Postel, Sunshine, and Chen [1981]; and in Hinden, Haverty, and Sheltzer [1983]. Shoch [1978] presents issues in internetwork naming and addressing. Boggs *et al.* [1980] describes the internet developed at Xerox PARC, an alternative to the TCP/IP internet we will examine. Cheriton [1983] describes internetworking as it relates to the V-system.

EXERCISES

- 3.1 What processors have been used as routers in the connected Internet? Does the size and speed of early router hardware surprise you? Why?
- 3.2 Approximately how many networks comprise the internet at your site? Approximately how many routers?
- 3.3 Consider the internal structure of the example internet shown in Figure 3.3b. Which routers are most crucial? Why?
- 3.4 Changing the information in a router can be tricky because it is impossible to change all routers simultaneously. Investigate algorithms that guarantee to either install a change on a set of computers or install it on none.
- 3.5 In an internet, routers periodically exchange information from their routing tables, making it possible for a new router to appear and begin routing packets. Investigate the algorithms used to exchange routing information.
- 3.6 Compare the organization of a TCP/IP internet to the style of internet designed by Xerox Corporation.

Classful Internet Addresses

4.1 Introduction

The previous chapter defines a TCP/IP internet as a virtual network built by interconnecting physical networks with routers. This chapter discusses addressing, an essential ingredient that helps TCP/IP software hide physical network details and makes the resulting internet appear to be a single, uniform entity.

4.2 Universal Identifiers

A communication system is said to supply *universal communication service* if it allows any host computer to communicate with any other host. To make our communication system universal, it needs a globally accepted method of identifying each computer that attaches to it.

Often, host identifiers are classified as *names*, *addresses*, or *routes*. Shoch [1978] suggests that a name identifies *what* an object is, an address identifies *where* it is, and a route tells *how* to get there[†]. Although these definitions are intuitive, they can be misleading. Names, addresses, and routes really refer to successively lower level representations of host identifiers. In general, people usually prefer pronounceable names to identify machines, while software works more efficiently with compact representations of identifiers that we think of as addresses. Either could have been chosen as the TCP/IP universal host identifiers. The decision was made to standardize on compact, binary addresses that make computations such as the selection of a route efficient. For now, we will discuss only binary addresses, postponing until later the questions of how to map between binary addresses and pronounceable names, and how to use addresses for routing.

[†]An identifier that specifies where an object can be found is also called a *locator*.

4.3 The Original Classful Addressing Scheme

Think of an internet as a large network like any other physical network. The difference, of course, is that the internet is a virtual structure, imagined by its designers, and implemented entirely in software. Thus, the designers are free to choose packet formats and sizes, addresses, delivery techniques, and so on; nothing is dictated by hardware. For addresses, the designers of TCP/IP chose a scheme analogous to physical network addressing in which each host on the internet is assigned a 32-bit integer address called its *internet address* or *IP address*. The clever part of internet addressing is that the integers are carefully chosen to make routing efficient. Specifically, an IP address encodes the identification of the network to which a host attaches as well as the identification of a unique host on that network. We can summarize:

Each host on a TCP/IP internet is assigned a unique 32-bit internet address that is used in all communication with that host.

The details of IP addresses help clarify the abstract ideas. For now, we give a simplified view and expand it later. In the simplest case, each host attached to an internet is assigned a 32-bit universal identifier as its internet address. A prefix of an IP address identifies a network. That is, the IP addresses in all hosts on a given network share a common prefix.

Conceptually, each address is a pair (*netid*, *hostid*), where *netid* identifies a network, and *hostid* identifies a host on that network. In practice, however, the partition into prefix and suffix is not uniform throughout the entire internet because the designers did not specify a single boundary. In the original addressing scheme, which is known as *classful*, each IP address had one of the first three forms shown in Figure 4.1†.

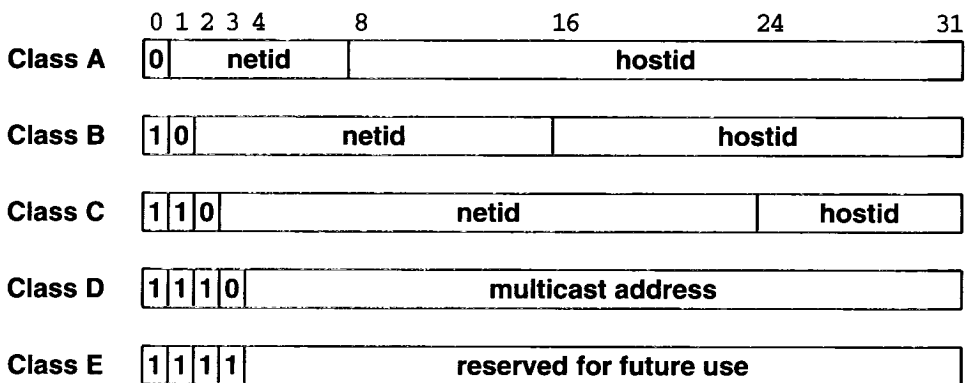


Figure 4.1 The five forms of Internet (IP) addresses used with the original classful addressing scheme. The three primary classes, A, B and C, can be distinguished by the first three bits.

†The fourth form, reserved for internet multicasting, will be described later; for now, we will restrict our comments to the forms that specify addresses of individual objects.

In the classful addressing scheme, each address is said to be *self-identifying* because the boundary between prefix and suffix can be computed from the address alone, without reference to external information. In particular, the class of an address can be determined from the three high-order bits, with two bits being sufficient to distinguish among the three primary classes. Class A addresses, used for the handful of networks that have more than 2^{16} (i.e., 65,536) hosts, devote 7 bits to netid and 24 bits to hostid. Class B addresses, used for intermediate size networks that have between 2^8 (i.e., 256) and 2^{16} hosts, allocate 14 bits to the netid and 16 bits to the hostid. Finally, class C addresses, used for networks that have less than 2^8 hosts, allocate 21 bits to the netid and only 8 bits to the hostid. Note that the IP address was originally defined in such a way that it was possible to extract the hostid or netid portions quickly. Efficiency was especially important for routers, which use the netid portion of an address when deciding where to send a packet. We will return to the discussion of efficient route lookup after examining recent changes and extensions to the addressing scheme.

4.4 Addresses Specify Network Connections

To simplify the discussion, we said that an internet address identifies a host, but that is not strictly accurate. Consider a router that attaches to two physical networks. How can we assign a single IP address if the address encodes a network identifier as well as a host identifier? In fact, we cannot. When conventional computers have two or more physical connections they are called *multi-homed hosts*. Multi-homed hosts and routers require multiple IP addresses. Each address corresponds to one of the machine's network connections. Looking at multi-homed hosts leads to the following important idea:

Because IP addresses encode both a network and a host on that network, they do not specify an individual computer, but a connection to a network.

Thus, a router connecting n networks has n distinct IP addresses, one for each network connection.

4.5 Network And Directed Broadcast Addresses

We have already cited the major advantage of encoding network information in internet addresses: it makes efficient routing possible. Another advantage is that internet addresses can refer to networks as well as hosts. By convention, hostid 0 is never assigned to an individual host. Instead, an IP address with hostid portion equal to zero is used to refer to the network itself. In summary:

Internet addresses can be used to refer to networks as well as individual hosts. By convention, an address that has all bits of the hostid equal to 0 is reserved to refer to the network.

Another significant advantage of the internet addressing scheme is that it includes a *directed broadcast address* that refers to all hosts on the network. According to the standard, any address with the hostid consisting of all 1s is reserved for directed broadcast†. When a packet is sent to such an address, a single copy of the packet is transferred across the internet from the source. Routers along the path use the netid portion of the address when choosing a path; they do not look at the host portion. Once the packet reaches a router attached to the final network, that router examines the host portion of the address to determine how to deliver the packet. If it finds all 1s, the router broadcasts the packet to all hosts on the network.

On many network technologies (e.g., Ethernet), broadcasting is as efficient as unicast transmission; on others, broadcasting is supported by the network software, but requires substantially more delay than single transmission. Some network hardware does not support broadcast at all. Thus, having an IP directed broadcast address does not guarantee the availability or efficiency of broadcast delivery. In summary,

IP addresses can be used to specify a directed broadcast in which a packet is sent to all computers on a network; such addresses map to hardware broadcast, if available. By convention, a directed broadcast address has a valid netid and has a hostid with all bits set to 1.

4.6 Limited Broadcast

The broadcast address we just described is known as *directed* because it contains both a valid network ID and the broadcast hostid. A directed broadcast address can be interpreted unambiguously at any point in an internet because it uniquely identifies the target network in addition to specifying broadcast on that network. Directed broadcast addresses provide a powerful (and somewhat dangerous) mechanism that allows a remote system to send a single packet that will be broadcast on the specified network.

From an addressing point of view, the chief disadvantage of directed broadcast is that it requires knowledge of the network address. Another form of broadcast address, called a *limited broadcast address* or *local network broadcast address*, provides a broadcast address for the local network independent of the assigned IP address. The local broadcast address consists of thirty-two 1s (hence, it is sometimes called the “all 1s” broadcast address). A host may use the limited broadcast address as part of a start-up procedure before it learns its IP address or the IP address prefix for the local network. Once the host learns the correct IP address for the local network, however, it should use directed broadcast.

†Unfortunately, an early release of TCP/IP code that accompanied Berkeley UNIX incorrectly used all zeroes for broadcast. Because the error still survives, TCP/IP software often includes an option that allows a site to use all zeroes for directed broadcast.

As a general rule, TCP/IP protocols restrict broadcasting to the smallest possible set of machines. We will see how this rule affects multiple networks that share addresses in the chapter on subnet addressing.

4.7 Interpreting Zero To Mean “This”

We have seen that a field consisting of *1*s can be interpreted to mean “all,” as in “all hosts” on a network. In general, internet software interprets fields consisting of *0*s to mean “this.” The interpretation appears throughout the literature. Thus, an IP address with hostid *0* refers to “this” host, and an internet address with network ID *0* refers to “this” network. Of course, it is only meaningful to use such an address in a context where it can be interpreted unambiguously. For example, if a machine receives a packet in which the netid portion of the destination address is *0* and the hostid portion of the destination address matches its address, the receiver interprets the netid field to mean “this” network (i.e., the network over which the packet arrived).

Using netid *0* is especially important in those cases where a host wants to communicate over a network but does not yet know the network IP address. The host uses network ID *0* temporarily, and other hosts on the network interpret the address as meaning “this” network. In most cases, replies will have the network address fully specified, allowing the original sender to record it for future use. Chapters 9 and 23 will discuss in detail mechanisms a host can use to determine the network ID of the local network.

4.8 Subnet And Supernet Extensions

The addressing scheme described so far requires a unique network prefix for each physical network. Although that was, indeed, the original plan, it did not last long. In the 1980s as Local Area Network technologies became increasingly popular, it became apparent that requiring a unique prefix for each physical network would exhaust the address space quickly. Consequently, an addressing extension was developed to conserve network prefixes. Known as *subnet addressing*, the scheme allows multiple physical networks to share a prefix.

In the 1990s, a second extension was devised that ignored the classful hierarchy and allowed the division between prefix and suffix to occur at an arbitrary point. Called *classless addressing* or *supernetting*, the scheme allows more complete utilization of the address space.

Chapter 10 will consider details of the subnet and supernet addressing extensions. For now, it is only important to know that the addressing scheme has been extended, and that the original classful scheme described in this chapter is no longer the most widely used.

4.9 IP Multicast Addresses

In addition to *unicast delivery*, in which a packet is delivered to a single computer, and *broadcast delivery*, in which a packet is delivered to all computers on a given network, the IP addressing scheme supports a special form of multipoint delivery known as *multicasting*, in which a packet is delivered to a specific subset of hosts. IP multicasting is especially useful for networks where the hardware technology supports multicast delivery. Chapter 17 discusses multicast addressing and delivery in detail. For now, it is sufficient to understand that Class *D* addresses are reserved for multicasting.

4.10 Weaknesses In Internet Addressing

Encoding network information in an internet address does have some disadvantages. The most obvious disadvantage is that addresses refer to network connections, not to the host computer:

If a host computer moves from one network to another, its IP address must change.

To understand the consequences, consider a traveler who wishes to disconnect his or her personal computer, carry it along on a trip, and reconnect it to the Internet after reaching the destination. The personal computer cannot be assigned a permanent IP address because an IP address identifies the network to which the machine attaches. Chapter 19 shows how the IP addressing scheme makes *mobility* a complex problem.

Another weakness of the classful addressing scheme is that when any class *C* network grows to more than 255 hosts, it must have its address changed to a class *B* address. While this may seem like a minor problem, changing network addresses can be incredibly time-consuming and difficult to debug. Because most software is not designed to handle multiple addresses for the same physical network, administrators cannot plan a smooth transition in which they introduce new addresses slowly. Instead, they must abruptly stop using one network address, change the addresses of all machines, and then resume communication using the new network address.

The most important flaw in the internet addressing scheme will not become fully apparent until we examine routing. However, its importance warrants a brief introduction here. We have suggested that routing will be based on internet addresses, with the netid portion of an address used to make routing decisions. Consider a host with two connections to the internet. We know that such a host must have more than one IP address. The following is true:

Because routing uses the network portion of the IP address, the path taken by packets traveling to a host with multiple IP addresses depends on the address used.

The implications are surprising. Humans think of each host as a single entity and want to use a single name. They are often surprised to find that they must learn more than one name and even more surprised to find that packets sent using multiple names can behave differently.

Another surprising consequence of the internet addressing scheme is that merely knowing one IP address for a destination may not be sufficient; it may be impossible to reach the destination using that address. Consider the example internet shown in Figure 4.2. In the figure, two hosts, *A* and *B*, both attach to network 1, and usually communicate directly using that network. Thus, users on host *A* should normally refer to host *B* using IP address I_3 . An alternate path from *A* to *B* exists through router *R*, and is used whenever *A* sends packets to IP address I_5 (*B*'s address on network 2). Now suppose *B*'s connection to network 1 fails, but the machine itself remains running (e.g., a wire breaks between *B* and network 1). Users on *A* who specify IP address I_3 cannot reach *B*, although users who specify address I_5 can. These problems with naming and addressing will arise again in later chapters when we consider routing and name binding.

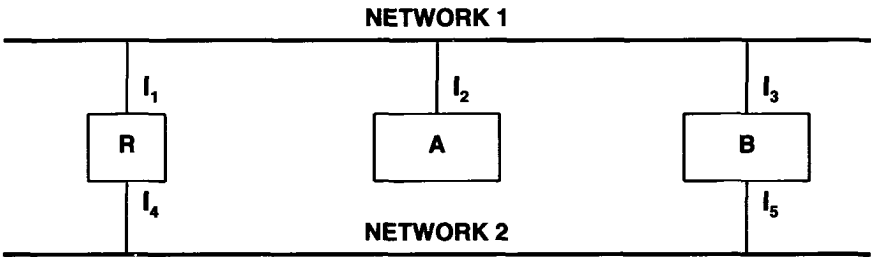


Figure 4.2 An example internet with a multi-homed host, *B*, that demonstrates a disadvantage of the IP addressing scheme. If interface I_3 becomes disconnected, *A* must use address I_5 to reach *B*, sending packets through router *R*.

4.11 Dotted Decimal Notation

When communicated to humans, either in technical documents or through application programs, IP addresses are written as four decimal integers separated by decimal points, where each integer gives the value of one octet of the IP address†. Thus, the 32-bit internet address

10000000 00001010 00000010 00011110

is written

128.10.2.30

†Dotted decimal notation is sometimes called *dotted quad notation*.

We will use dotted decimal notation when expressing IP addresses throughout the remainder of this text. Indeed, most TCP/IP software that displays or requires a human to enter an IP address uses dotted decimal notation. For example, the UNIX *netstat* command, which displays information about routes and connections, and application programs such as *telnet* and *ftp* all use dotted decimal notation when accepting or displaying IP addresses. Thus, when classful addressing is used, it is helpful to understand the relationship between IP address classes and dotted decimal numbers. The table in Figure 4.3 summarizes the range of values for each class.

Class	Lowest Address	Highest Address
A	1.0.0.0	126.0.0.0
B	128.1.0.0	191.255.0.0
C	192.0.1.0	223.255.255.0
D	224.0.0.0	239.255.255.255
E	240.0.0.0	255.255.255.254

Figure 4.3 The range of dotted decimal values that correspond to each IP address class. Some values are reserved for special purposes.

4.12 Loopback Address

The table in Figure 4.3 shows that not all possible addresses have been assigned to classes. In particular, the network prefix 127.0.0.0, a value from the class A range, is reserved for *loopback*, and is intended for use in testing TCP/IP and for inter-process communication on the local computer. When any program uses the loopback address as a destination, the protocol software in the computer processes the data without sending traffic across any network. The literature explicitly states that a packet sent to a network 127 address should never appear on any network. Furthermore, a host or router should never propagate routing or reachability information for network number 127; it is not a network address.

4.13 Summary Of Special Address Conventions

In practice, IP uses only a few combinations of 0s (“this”) or 1s (“all”). Figure 4.4 lists the possibilities.

all 0s		This host ¹
all 0s	host	Host on this net ¹
all 1s		Limited broadcast (local net) ²
net	all 1s	Directed broadcast for net ²
127	anything (often 1)	Loopback ³

Notes: ¹ Allowed only at system startup and is never a valid destination address.
² Never a valid source address.
³ Should never appear on a network.

Figure 4.4 Special forms of IP addresses, including valid combinations of 0s (“this”), 1s (“all”). The length of the net portion of a directed broadcast depends on the network address class.

As the notes in the figure mention, using all 0s for the network is only allowed during the bootstrap procedure. Doing so allows a machine to communicate temporarily. Once the machine learns its correct network and IP address, it must not use network prefix 0.

4.14 Internet Addressing Authority

Each network address prefix used within a given TCP/IP internet must be unique. An organization that uses TCP/IP technology to build a completely private internet (i.e., one that is not connected to the global Internet) can assign address prefixes without considering the assignments made by other organizations. However, an organization that connects to the global Internet must not use address prefixes assigned to another organization. To ensure that the network portion of an address is unique in the global internet, all Internet addresses are assigned by a central authority. Originally, the *Internet Assigned Number Authority (IANA)* had control over numbers assigned, and set the policy. From the time the Internet began until the fall of 1998, a single individual, Jon Postel, ran the IANA and assigned addresses. In late 1998, after Jon’s untimely death, a new organization was created to handle address assignment. Named the *Internet Corporation For Assigned Names and Numbers (ICANN)*, the organization sets policy and assigns values for names and other constants used in protocols as well as addresses.

In the original classful scheme, the Internet authority chose an address appropriate to the size of the network. A class C number was assigned to a network with a small number of attached computers (less than 255); class B numbers were reserved for larger networks. Finally, a network needed to have more than 65,535 hosts before it could obtain a class A number. The address space was skewed because most networks are small, fewer are of medium size, and only a handful are gigantic.

Most organizations never interact with the central authority directly. Instead, to connect its networks to the global Internet, an organization usually contracts with a local *Internet Service Provider (ISP)*. In addition to providing a connection between the organization and the rest of the Internet, an ISP obtains a valid address prefix for each of the customer's networks. Many local ISPs are, in fact, customers of larger ISPs — when a customer requests an address prefix, the local ISP merely obtains a prefix from a larger ISP. Thus, only the largest ISPs need to contact ICANN.

Note that the central authority only assigns the network portion of an address; once an organization obtains a prefix for a network, the organization can choose how to assign a unique suffix to each host on the network without contacting the central authority. Furthermore, remember that it is only essential for the central authority to assign IP addresses for networks that are (or will be) attached to the global Internet.

4.15 Reserved Address Prefixes

We said that as long as it never connects to the outside world, an individual corporation has responsibility for assigning unique network addresses within its TCP/IP internet. Indeed, many corporate groups that use TCP/IP protocols do assign internet addresses on their own. For example, the network address 9.0.0.0 has been assigned to IBM Corporation, and address 12.0.0.0 has been assigned to AT&T. If an organization decides to use TCP/IP protocols on two of their networks with no connections to the global Internet, the organization can choose to assign addresses 9.0.0.0 and 12.0.0.0 to their local networks.

Experience has shown, however, that it is unwise to create a private internet using the same network addresses as the global Internet because most sites eventually connect to the Internet and doing so may cause problems when trying to exchange software with other sites. To avoid addressing conflicts between addresses used on private internets and addresses used on the global Internet, the IETF reserved several address prefixes, and recommends using them on private internets. Because the set of reserved prefixes includes both classful and classless values, they are described in Chapter 10.

4.16 An Example

To clarify the IP addressing scheme, consider an example of two networks in the Computer Science Department at Purdue University as they were connected to the Internet in the mid-1980s. Figure 4.5 shows the network addresses, and illustrates how routers interconnect the networks.

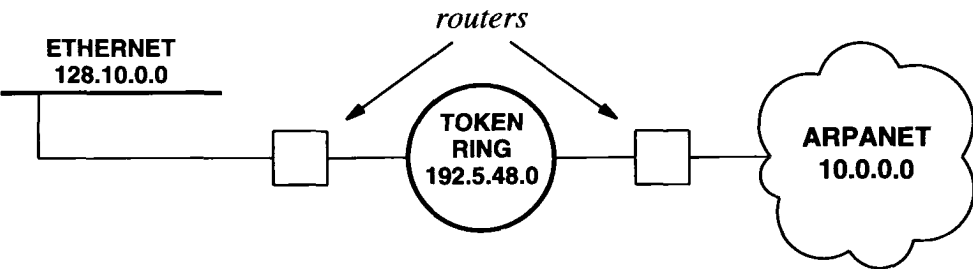


Figure 4.5 The logical connection of two networks to the Internet backbone.
Each network has been assigned an IP address.

The example shows three networks and the network numbers they have been assigned: the ARPANET (10.0.0.0), an Ethernet (128.10.0.0), and a token ring network (192.5.48.0). According to the table in Figure 4.3, the addresses have classes *A*, *B*, and *C*, respectively.

Figure 4.6 shows the same networks with host computers attached and Internet addresses assigned to each network connection.

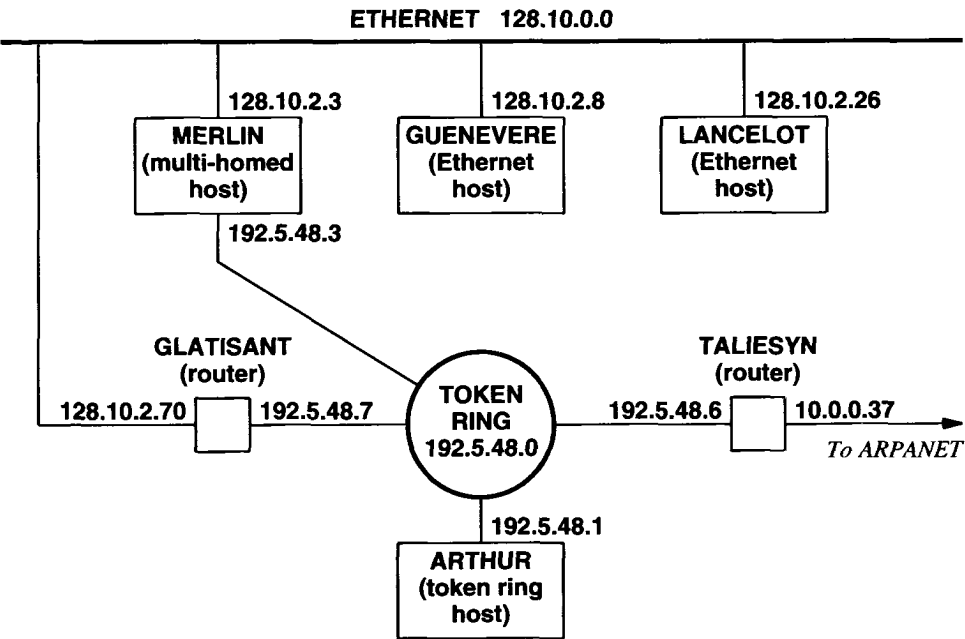


Figure 4.6 Example IP address assignment for routers and hosts attached to the three networks in the previous figure.

In the figure, four hosts labeled *Arthur*, *Merlin*, *Guenevere*, and *Lancelot*, attach to the networks, *Taliesyn* is a router that connects the ARPANET and the token ring network, and *Glatissant* is a router that connects the token ring network to the Ethernet. Host *Merlin* has connections to both the Ethernet and the token ring network, so it can reach destinations on either network directly. Although a multi-homed host like *Merlin* can be configured to route packets between the two nets, most sites use dedicated computers as routers to avoid overloading conventional computer systems with the processing required for routing. In the figure, a dedicated router, *Glatissant*, performs the task of routing traffic between the Ethernet and token ring networks. (Note: actual traffic between these two networks was higher than this configuration suggests because the figure only shows a few of the computers attached to the nets.)

As Figure 4.5 shows, an IP address must be assigned to each network connection. *Lancelot*, which connects only to the Ethernet, has been assigned 128.10.2.26 as its only IP address. *Merlin* has address 128.10.2.3 for its connection to the Ethernet and 192.5.48.3 for its connection to the token ring network. Whoever made the address assignment chose the same value for the low-order byte of each address. The addresses assigned to routers *Glatissant* and *Taliesyn* do not follow the convention. For example, *Taliesyn*'s addresses, 10.0.0.37 and 192.5.48.6, are two completely unrelated strings of digits. IP does not care whether any of the bytes in the dotted decimal form of a computer's addresses are the same or different. However, network technicians, managers, and administrators may need to use addresses for maintenance, testing, and debugging. Choosing to make all of a computer's addresses end with the same octet makes it easier for humans to remember or guess the address of a particular interface.

4.17 Network Byte Order

To create an internet that is independent of any particular vendor's machine architecture or network hardware, the software must define a standard representation for data. Consider what happens, for example, when software on one computer sends a 32-bit binary integer to another computer. The physical transport hardware moves the sequence of bits from the first machine to the second without changing the order. However, not all architectures store 32-bit integers in the same way. On some (called *Little Endian*), the lowest memory address contains the low-order byte of the integer. On others (called *Big Endian*), the lowest memory address holds the high-order byte of the integer. Still others store integers in groups of 16-bit words, with the lowest addresses holding the low-order word, but with bytes swapped. Thus, direct copying of bytes from one machine to another may change the value of the number.

Standardizing byte-order for integers is especially important in an internet because internet packets carry binary numbers that specify information like destination addresses and packet lengths. Such quantities must be understood by both the senders and receivers. The TCP/IP protocols solve the byte-order problem by defining a *network standard byte order* that all machines must use for binary fields in internet packets. Each host or router converts binary items from the local representation to network standard byte order before sending a packet, and converts from network byte order to the host-specific order when a packet arrives. Naturally, the user data field in a packet is

exempt from this standard because the TCP/IP protocols do not know what data is being carried — application programmers are free to format their own data representation and translation. When sending integer values, many application programmers do choose to follow the TCP/IP byte-order standards. Of course, users who merely invoke application programs never need to deal with the byte order problem directly.

The internet standard for byte order specifies that integers are sent with the most significant byte first (i.e., *Big Endian* style). If one considers the successive bytes in a packet as it travels from one machine to another, a binary integer in that packet has its most significant byte nearest the beginning of the packet and its least significant byte nearest the end of the packet. Many arguments have been offered about which data representation should be used, and the internet standard still comes under attack from time to time. In particular, proponents of change argue that although most computers were big endian when the standard was defined, most are now little endian. However, everyone agrees that having a standard is crucial, and the exact form of the standard is far less important.

4.18 Summary

TCP/IP uses 32-bit binary addresses as universal machine identifiers. Called Internet Protocol addresses or IP addresses, the identifiers are partitioned into two parts: a prefix identifies the network to which the computer attaches and the suffix provides a unique identifier for the computer on that network. The original IP addressing scheme is known as classful, with each prefix assigned to one of three primary classes. Leading bits define the class of an address; the classes are of unequal size. The classful scheme provides for 127 networks with over a million hosts each, thousands of networks with thousands of hosts each, and over a million networks with up to 254 hosts each. To make such addresses easier for humans to understand, they are written in dotted decimal notation, with the values of the four octets written in decimal, separated by decimal points.

Because the IP address encodes network identification as well as the identification of a specific host on that network, routing is efficient. An important property of IP addresses is that they refer to network connections. Hosts with multiple connections have multiple addresses. One advantage of the internet addressing scheme is that the form includes an address for a specific host, a network, or all hosts on a network (broadcast). The biggest disadvantage of the IP addressing scheme is that if a machine has multiple addresses, knowing one address may not be sufficient to reach it when no path exists to the specified interface (e.g., because a particular network is unavailable).

To permit the exchange of binary data among machines, TCP/IP protocols enforce a standard byte ordering for integers within protocol fields. A host must convert all binary data from its internal form to network standard byte order before sending a packet, and it must convert from network byte order to internal order upon receipt.

FOR FURTHER STUDY

The internet addressing scheme presented here can be found in Reynolds and Postel [RFC 1700]; further information can be found in Stahl, Romano, and Recker [RFC 1117].

Several important additions have been made to the Internet addressing scheme over the years; later chapters cover them in more detail. Chapter 10 discusses an important extension called *classless addressing* that permits the division between prefix and suffix to occur at an arbitrary bit position. In addition, Chapter 10 examines an essential part of the Internet address standard called *subnet addressing*. Subnet addressing allows a single network address to be used with multiple physical networks. Chapter 17 continues the exploration of IP addresses by describing how class D addresses are assigned for internet *multicast*.

Cohen [1981] explains bit and byte ordering, and introduces the terms “Big Endian” and “Little Endian.”

EXERCISES

- 4.1 Exactly how many class A, B, and C networks can exist? Exactly how many hosts can a network in each class have? Be careful to allow for broadcast as well as class D and E addresses.
- 4.2 A machine readable list of assigned addresses is sometimes called an internet *host table*. If your site has a host table, find out how many class A, B, and C network numbers have been assigned.
- 4.3 How many hosts are attached to each of the local area networks at your site? Does your site have any local area networks for which a class C address is insufficient?
- 4.4 What is the chief difference between the IP addressing scheme and the U.S. telephone numbering scheme?
- 4.5 A single central authority cannot manage to assign Internet addresses fast enough to accommodate the demand. Can you invent a scheme that allows the central authority to divide its task among several groups but still ensure that each assigned address is unique?
- 4.6 Does network standard byte order differ from your local machine’s byte order?
- 4.7 How many IP addresses would be needed to assign a unique IP address to every house in your country? the world? Is the IP address space sufficient?

5

Mapping Internet Addresses To Physical Addresses (ARP)

5.1 Introduction

We described the TCP/IP address scheme in which each host is assigned a 32-bit address, and said that an internet behaves like a virtual network, using only the assigned addresses when sending and receiving packets. We also reviewed several network hardware technologies, and noted that two machines on a given physical network can communicate *only if they know each other's physical network address*. What we have not mentioned is how a host or a router maps an IP address to the correct physical address when it needs to send a packet across a physical net. This chapter considers that mapping, showing how it is implemented for the two most common physical network address schemes.

5.2 The Address Resolution Problem

Consider two machines A and B that connect to the same physical network. Each has an assigned IP address I_A and I_B and a physical address P_A and P_B . The goal is to devise low-level software that hides physical addresses and allows higher-level programs to work only with internet addresses. Ultimately, however, communication must be carried out by physical networks using whatever physical address scheme the underlying network hardware supplies. Suppose machine A wants to send a packet to

machine B across a physical network to which they both attach, but A has only B 's internet address I_B . The question arises: how does A map that address to B 's physical address, P_B ?

Address mapping must be performed at each step along a path from the original source to the ultimate destination. In particular, two cases arise. First, at the last step of delivering a packet, the packet must be sent across one physical network to its final destination. The computer sending the packet must map the final destination's Internet address to the destination's physical address. Second, at any point along the path from the source to the destination other than the final step, the packet must be sent to an intermediate router. Thus, the sender must map the intermediate router's Internet address to a physical address.

The problem of mapping high-level addresses to physical addresses is known as the *address resolution problem* and has been solved in several ways. Some protocol suites keep tables in each machine that contain pairs of high-level and physical addresses. Others solve the problem by encoding hardware addresses in high-level addresses. Using either approach exclusively makes high-level addressing awkward at best. This chapter discusses two techniques for address resolution used by TCP/IP protocols and shows when each is appropriate.

5.3 Two Types Of Physical Addresses

There are two basic types of physical addresses, exemplified by the Ethernet, which has large, fixed physical addresses, and proNET, which has small, easily configured physical addresses. Address resolution is difficult for Ethernet-like networks, but easy for networks like proNET. We will consider the easy case first.

5.4 Resolution Through Direct Mapping

Consider a proNET token ring network. Recall from Chapter 2 that proNET uses small integers for physical addresses and allows the user to choose a hardware address when installing an interface board in a computer. The key to making address resolution easy with such network hardware lies in observing that as long as one has the freedom to choose both IP and physical addresses, they can be selected such that parts of them are the same. Typically, one assigns IP addresses with the hostid portion equal to 1, 2, 3, and so on, and then, when installing network interface hardware, selects a physical address that corresponds to the IP address. For example, the system administrator would select physical address 3 for a computer with the IP address 192.5.48.3 because 192.5.48.3 is a class C address with the host portion equal to 3.

For networks like proNET, computing a physical address from an IP address is trivial. The computation consists of extracting the host portion of the IP address. Extraction is computationally efficient on most architectures because it requires only a few machine instructions. The mapping is easy to maintain because it can be performed

without reference to external data. Finally, new computers can be added to the network without changing existing assignments or recompiling code.

Conceptually, choosing a numbering scheme that makes address resolution efficient means selecting a function f that maps IP addresses to physical addresses. The designer may be able to select a physical address numbering scheme as well, depending on the hardware. Resolving IP address I_A means computing

$$P_A = f(I_A)$$

We want the computation of f to be efficient. If the set of physical addresses is constrained, it may be possible to arrange efficient mappings other than the one given in the example above. For instance, when using IP over a connection-oriented network such as ATM, one cannot choose physical addresses. On such networks, one or more computers (servers) store pairs of addresses, where each pair contains an Internet address and the corresponding physical address. Typically, such servers store the pairs in a table in memory to speed searching. To guarantee efficient address resolution in such cases, software can use a conventional hash function to search the table. Exercise 5.1 suggests a related alternative.

5.5 Resolution Through Dynamic Binding

To understand why address resolution is difficult for some networks, consider Ethernet technology. Recall from Chapter 2 that each Ethernet interface is assigned a 48-bit physical address when the device is manufactured. As a consequence, when hardware fails and requires that an Ethernet interface be replaced, the machine's physical address changes. Furthermore, because the Ethernet address is 48 bits long, there is no hope it can be encoded in a 32-bit IP address[†].

Designers of TCP/IP protocols found a creative solution to the address resolution problem for networks like the Ethernet that have broadcast capability. The solution allows new hosts or routers to be added to the network without recompiling code, and does not require maintenance of a centralized database. To avoid maintaining a table of mappings, the designers chose to use a low-level protocol to bind addresses dynamically. Termed the *Address Resolution Protocol* (ARP), the protocol provides a mechanism that is both reasonably efficient and easy to maintain.

As Figure 5.1 shows, the idea behind dynamic resolution with ARP is simple: when host A wants to resolve IP address I_B , it broadcasts a special packet that asks the host with IP address I_B to respond with its physical address, P_B . All hosts, including B , receive the request, but only host B recognizes its IP address and sends a reply that contains its physical address. When A receives the reply, it uses the physical address to send the internet packet directly to B . We can summarize:

[†]Because direct mapping is more convenient and efficient than dynamic binding, the next generation of IP is being designed to allow 48-bit hardware addresses to be encoded in IP addresses.

The Address Resolution Protocol, ARP, allows a host to find the physical address of a target host on the same physical network, given only the target's IP address.

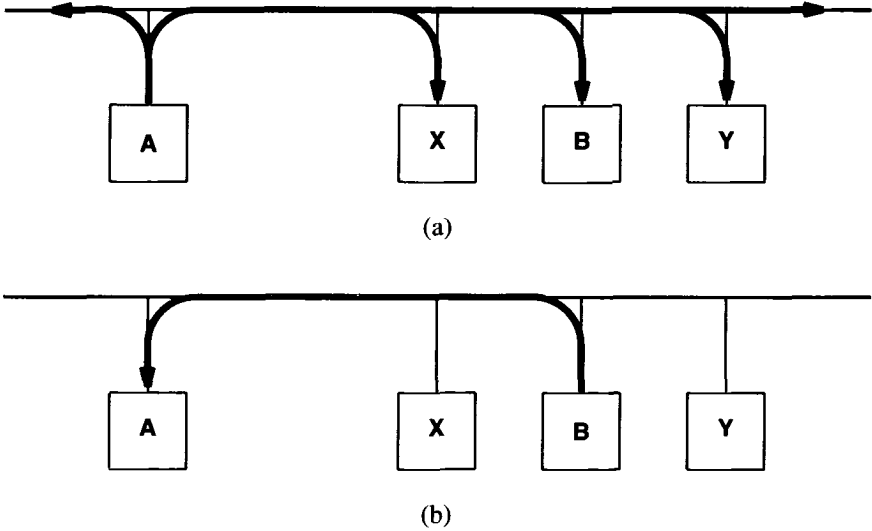


Figure 5.1 The ARP protocol. To determine P_B , B 's physical address, from I_B , its IP address, (a) host A broadcasts an ARP request containing I_B to all machines on the net, and (b) host B responds with an ARP reply that contains the pair (I_B, P_B) .

5.6 The Address Resolution Cache

It may seem silly that for A to send a packet to B it first sends a broadcast that reaches B . Or it may seem even sillier that A broadcasts the question, “how can I reach you?” instead of just broadcasting the packet it wants to deliver. But there is an important reason for the exchange. Broadcasting is far too expensive to be used every time one machine needs to transmit a packet to another because every machine on the network must receive and process the broadcast packet.

5.7 ARP Cache Timeout

To reduce communication costs, computers that use ARP maintain a cache of recently acquired IP-to-physical address bindings. That is, whenever a computer sends an ARP request and receives an ARP reply, it saves the IP address and corresponding hardware address information in its cache for successive lookups. When transmitting a packet, a computer always looks in its cache for a binding before sending an ARP request. If it finds the desired binding in its ARP cache, the computer need not broadcast on the network. Thus, when two computers on a network communicate, they begin with an ARP request and response, and then repeatedly transfer packets without using ARP for each one. Experience shows that because most network communication involves more than one packet transfer, even a small cache is worthwhile.

The ARP cache provides an example of *soft state*, a technique commonly used in network protocols. The name describes a situation in which information can become “stale” without warning. In the case of ARP, consider two computers, *A* and *B*, both connected to an Ethernet. Assume *A* has sent an ARP request, and *B* has replied. Further assume that after the exchange *B* crashes. Computer *A* will not receive any notification of the crash. Moreover, because it already has address binding information for *B* in its ARP cache, computer *A* will continue to send packets to *B*. The Ethernet hardware provides no indication that *B* is not on-line because Ethernet does not have guaranteed delivery. Thus, *A* has no way of knowing when information in its ARP cache has become incorrect.

To accommodate soft state, responsibility for correctness lies with the owner of the information. Typically, protocols that implement soft state use timers, with the state information being deleted when the timer expires. For example, whenever address binding information is placed in an ARP cache, the protocol requires a timer to be set, with a typical timeout being 20 minutes. When the timer expires, the information must be removed. After removal there are two possibilities. If no further packets are sent to the destination, nothing occurs. If a packet must be sent to the destination and there is no binding present in the cache, the computer follows the normal procedure of broadcasting an ARP request and obtaining the binding. If the destination is still reachable, the binding will again be placed in the ARP cache. If not, the sender will discover that the destination is off-line.

The use of soft state in ARP has advantages and disadvantages. The chief advantage arises from autonomy. First, a computer can determine when information in its ARP cache should be revalidated independent of other computers. Second, a sender does not need successful communication with the receiver or a third party to determine that a binding has become invalid; if a target does not respond to an ARP request, the sender will declare the target to be down. Third, the scheme does not rely on network hardware to provide reliable transfer. The chief disadvantage of soft state arises from delay — if the timer interval is N seconds, a sender may not detect that a receiver has crashed until N seconds elapse.

5.8 ARP Refinements

Several refinements of ARP have been included in the protocol. First, observe that if host *A* is about to use ARP because it needs to send to *B*, there is a high probability that host *B* will need to send to *A* in the near future. To anticipate *B*'s need and avoid extra network traffic, *A* includes its IP-to-physical address binding when sending *B* a request. *B* extracts *A*'s binding from the request, saves the binding in its ARP cache, and then sends a reply to *A*. Second, notice that because *A* broadcasts its initial request, all machines on the network receive it and can extract and update *A*'s IP-to-physical address binding in their cache. Third, when a computer has its host interface replaced, (e.g., because the hardware has failed) its physical address changes. Other computers on the net that have stored a binding in their ARP cache need to be informed so they can change the entry. The computer can notify others of a new address by sending an ARP broadcast when it boots.

The following rule summarizes refinements:

The sender's IP-to-physical address binding is included in every ARP broadcast; receivers update the IP-to-physical address binding information in their cache before processing an ARP packet.

5.9 Relationship Of ARP To Other Protocols

ARP provides one possible mechanism to map from IP addresses to physical addresses; we have already seen that some network technologies do not need it. The point is that ARP would be completely unnecessary if we could make all network hardware recognize IP addresses. Thus, ARP merely imposes a new address scheme on top of whatever low-level address mechanism the hardware uses. The idea can be summarized:

ARP is a low-level protocol that hides the underlying network physical addressing, permitting one to assign an arbitrary IP address to every machine. We think of ARP as part of the physical network system, and not as part of the internet protocols.

5.10 ARP Implementation

Functionally, ARP is divided into two parts. The first part maps an IP address to a physical address when sending a packet, and the second part answers requests from other machines. Address resolution for outgoing packets seems straightforward, but small details complicate an implementation. Given a destination IP address the software consults its ARP cache to see if it knows the mapping from IP address to physical address.

If it does, the software extracts the physical address, places the data in a frame using that address, and sends the frame. If it does not know the mapping, the software must broadcast an ARP request and wait for a reply.

Broadcasting an ARP request to find an address mapping can become complex. The target machine can be down or just too busy to accept the request. If so, the sender may not receive a reply or the reply may be delayed. Because the Ethernet is a best-effort delivery system, the initial ARP broadcast request can also be lost (in which case the sender should retransmit, at least once). Meanwhile, the host must store the original outgoing packet so it can be sent once the address has been resolved[†]. In fact, the host must decide whether to allow other application programs to proceed while it processes an ARP request (most do). If so, the software must handle the case where an application generates additional ARP requests for the same address without broadcasting multiple requests for a given target.

Finally, consider the case where machine *A* has obtained a binding for machine *B*, but then *B*'s hardware fails and is replaced. Although *B*'s address has changed, *A*'s cached binding has not, so *A* uses a nonexistent hardware address, making successful reception impossible. This case shows why it is important to have ARP software treat its table of bindings as a cache and remove entries after a fixed period. Of course, the timer for an entry in the cache must be reset whenever an ARP broadcast arrives containing the binding (but it is not reset when the entry is used to send a packet).

The second part of the ARP code handles ARP packets that arrive from the network. When an ARP packet arrives, the software first extracts the sender's IP address and hardware address pair, and examines the local cache to see if it already has an entry for the sender. If a cache entry exists for the given IP address, the handler updates that entry by overwriting the physical address with the physical address obtained from the packet. The receiver then processes the rest of the ARP packet.

A receiver must handle two types of incoming ARP packets. If an ARP request arrives, the receiving machine must see if it is the target of the request (i.e., some other machine has broadcast a request for the receiver's physical address). If so, the ARP software forms a reply by supplying its physical hardware address, and sends the reply directly back to the requester. The receiver also adds the sender's address pair to its cache if the pair is not already present. If the IP address mentioned in the ARP request does not match the local IP address, the packet is requesting a mapping for some other machine on the network and can be ignored.

The other interesting case occurs when an ARP reply arrives. Depending on the implementation, the handler may need to create a cache entry, or the entry may have been created when the request was generated. In any case, once the cache has been updated, the receiver tries to match the reply with a previously issued request. Usually, replies arrive in response to a request, which was generated because the machine has a packet to deliver. Between the time a machine broadcasts its ARP request and receives the reply, application programs or higher-level protocols may generate additional requests for the same address; the software must remember that it has already sent a request and not send more. Usually, ARP software places the additional packets on a queue. Once the reply arrives and the address binding is known, the ARP software re-

[†]If the delay is significant, the host may choose to discard the outgoing packet(s).

moves packets from the queue, places each packet in a frame, and uses the address binding to fill in the physical destination address. If it did not previously issue a request for the IP address in the reply, the machine updates the sender’s entry in its cache, and then simply stops processing the packet.

5.11 ARP Encapsulation And Identification

When ARP messages travel from one machine to another, they must be carried in physical frames. Figure 5.2 shows that the ARP message is carried in the data portion of a frame.

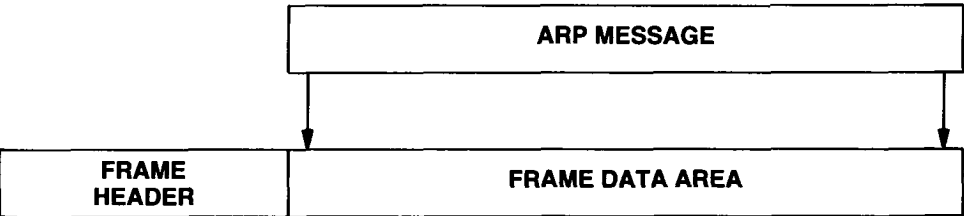


Figure 5.2 An ARP message encapsulated in a physical network frame.

To identify the frame as carrying an ARP message, the sender assigns a special value to the type field in the frame header, and places the ARP message in the frame’s data field. When a frame arrives at a computer, the network software uses the frame type to determine its contents. In most technologies, a single type value is used for all frames that carry an ARP message — network software in the receiver must further examine the ARP message to distinguish between ARP requests and ARP replies. For example, on an Ethernet, frames carrying ARP messages have a type field of 0806₁₆. This is a standard value assigned by the authority for Ethernet; other network hardware technologies use other values.

5.12 ARP Protocol Format

Unlike most protocols, the data in ARP packets does not have a fixed-format header. Instead, to make ARP useful for a variety of network technologies, the length of fields that contain addresses depend on the type of network. However, to make it possible to interpret an arbitrary ARP message, the header includes fixed fields near the beginning that specify the lengths of the addresses found in succeeding fields. In fact, the ARP message format is general enough to allow it to be used with arbitrary physical addresses and arbitrary protocol addresses. The example in Figure 5.3 shows the 28-octet ARP message format used on Ethernet hardware (where physical addresses are

48-bits or 6 octets long), when resolving IP protocol addresses (which are 4 octets long).

Figure 5.3 shows an ARP message with 4 octets per line, a format that is standard throughout this text. Unfortunately, unlike most of the remaining protocols, the variable-length fields in ARP packets do not align neatly on 32-bit boundaries, making the diagram difficult to read. For example, the sender’s hardware address, labeled *SENDER HA*, occupies 6 contiguous octets, so it spans two lines in the diagram.

0	8	16	24	31
HARDWARE TYPE		PROTOCOL TYPE		
HLEN	PLEN	OPERATION		
SENDER HA (octets 0-3)				
SENDER HA (octets 4-5)		SENDER IP (octets 0-1)		
SENDER IP (octets 2-3)		TARGET HA (octets 0-1)		
TARGET HA (octets 2-5)				
TARGET IP (octets 0-3)				

Figure 5.3 An example of the ARP/RARP message format when used for IP-to-Ethernet address resolution. The length of fields depends on the hardware and protocol address lengths, which are 6 octets for an Ethernet address and 4 octets for an IP address.

Field *HARDWARE TYPE* specifies a hardware interface type for which the sender seeks an answer; it contains the value *1* for Ethernet. Similarly, field *PROTOCOL TYPE* specifies the type of high-level protocol address the sender has supplied; it contains *0800₁₆* for IP addresses. Field *OPERATION* specifies an ARP request (*1*), ARP response (*2*), RARP† request (*3*), or RARP response (*4*). Fields *HLEN* and *PLEN* allow ARP to be used with arbitrary networks because they specify the length of the hardware address and the length of the high-level protocol address. The sender supplies its hardware address and IP address, if known, in fields *SENDER HA* and *SENDER IP*.

When making a request, the sender also supplies the target hardware address (RARP) or target IP address (ARP), using fields *TARGET HA* or *TARGET IP*. Before the target machine responds, it fills in the missing addresses, swaps the target and sender pairs, and changes the operation to a reply. Thus, a reply carries the IP and hardware addresses of the original requester, as well as the IP and hardware addresses of the machine for which a binding was sought.

†The next chapter describes RARP, another protocol that uses the same message format.

5.13 Summary

IP addresses are assigned independent of a machine's physical hardware address. To send an internet packet across a physical net from one computer to another, the network software must map the IP address into a physical hardware address and use the hardware address to transmit the frame. If hardware addresses are smaller than IP addresses, a direct mapping can be established by having the machine's physical address encoded in its IP address. Otherwise, the mapping must be performed dynamically. The Address Resolution Protocol (ARP) performs dynamic address resolution, using only the low-level network communication system. ARP permits machines to resolve addresses without keeping a permanent record of bindings.

A machine uses ARP to find the hardware address of another machine by broadcasting an ARP request. The request contains the IP address of the machine for which a hardware address is needed. All machines on a network receive an ARP request. If the request matches a machine's IP address, the machine responds by sending a reply that contains the needed hardware address. Replies are directed to one machine; they are not broadcast.

To make ARP efficient, each machine caches IP-to-physical address bindings. Because internet traffic tends to consist of a sequence of interactions between pairs of machines, the cache eliminates most ARP broadcast requests.

FOR FURTHER STUDY

The address resolution protocol used here is given by Plummer [RFC 826] and has become a TCP/IP internet protocol standard. Dalal and Printis [1981] describes the relationship between Ethernet and IP addresses, and Clark [RFC 814] discusses addresses and bindings in general. Parr [RFC 1029] discusses fault tolerant address resolution. Kirkpatrick and Recker [RFC 1166] specifies values used to identify network frames in the Internet Numbers document. Volume 2 of this text presents an example ARP implementation, and discusses the caching policy.

EXERCISES

- 5.1 Given a small set of physical addresses (positive integers), can you find a function f and an assignment of IP addresses such that f maps the IP addresses 1-to-1 onto the physical addresses and computing f is efficient? (Hint: look at the literature on perfect hashing).
- 5.2 In what special cases does a host connected to an Ethernet not need to use ARP or an ARP cache before transmitting an IP datagram?

- 5.3 One common algorithm for managing the ARP cache replaces the least recently used entry when adding a new one. Under what circumstances can this algorithm produce unnecessary network traffic?
- 5.4 Read the standard carefully. Should ARP update the cache if an old entry already exists for a given IP address? Why or why not?
- 5.5 Should ARP software modify the cache even when it receives information without specifically requesting it? Why or why not?
- 5.6 Any implementation of ARP that uses a fixed-size cache can fail when used on a network that has many hosts and much ARP traffic. Explain how.
- 5.7 ARP is often cited as a security weakness. Explain why.
- 5.8 Suppose an (incorrect) ARP implementation does not remove cache entries if they are frequently used. Explain what can happen if the hardware address field in an ARP response becomes corrupted during transmission.
- 5.9 Suppose machine C receives an ARP request sent from A looking for target B , and suppose C has the binding from I_B to P_B in its cache. Should C answer the request? Explain.
- 5.10 How can a workstation use ARP when it boots to find out if any other machine on the network is impersonating it? What are the disadvantages of the scheme?
- 5.11 Explain how sending IP packets to nonexistent addresses on a remote Ethernet can generate excess broadcast traffic on that network.